Intent-Governed Tool Authorization for AI Agents

Genliang Zhu
Accentrust
Georgia Institute of Technology

Chu Wang
Accentrust
University of Illinois Urbana-Champaign

arXiv:2606.22916v4 [cs.AI] 18 Sep 2026

Abstract

Tool-using AI agents often use credentials broader than a user's current request. We present an *intent-governed tool-authorization layer* that derives a short-lived certificate from a trusted request, narrows the authorized tool manifest, and checks proposed tool and payload effects before execution. It cannot exceed static policy; request confinement also requires certificate fidelity and sound effect bounds. We evaluate it over OpenPort with endpoint tests, 176 runtime-backed synthetic tasks, real-model pilots, 306 end-to-end trials, and a 36-trial benchmark-shaped external subset. In the deterministic comparison, reference certificates reduce the archived composite exposure-or-path indicator from 1.0000 to 0. Across end-to-end model runs, the combined path records no completed unsafe executions; unsafe accepted authority remains 0.0909–0.2727, entirely as non-executed drafts. Trace-backed normalization removes this residual at substantial utility cost. The results support static-policy non-expansion and identify certificate precision as the principal remaining bottleneck.

Keywords: AI agents, authorization, access control, least privilege, prompt injection.

I. Introduction

AI agents are moving from conversational assistance to operational action. A deployed agent can inspect records, update state, export data, coordinate workflows, and act across application boundaries. In such settings, tool access is no longer a convenience feature of a language model interface. It is an application security boundary.

The access-control question normally asked by an application gateway is credential-centric: does this token, app, or integration have permission to call this endpoint? Credential-centric authorization is necessary. It supports revocation, least privilege, tenant isolation, and accountability. But agentic tool use introduces a second question that traditional API authorization rarely models: *is the proposed tool call justified by what the user currently asked the agent to do?*

This distinction matters because the same credential can be legitimately broad while a particular user request is narrow. An agent may have both read and export scopes because the organization approves those capabilities for some workflows. Yet if the user asks "summarize recent records for this period," exposing a bulk export tool to the model is unnecessary and potentially dangerous. If the model later proposes an export because an external document, retrieved note, or malicious tool description influenced it, credential-only authorization cannot distinguish the overbroad effect from the user's original summary request. Likewise, a model may construct a plausible update payload that touches a different resource than the user named, or a delete call that exceeds the intended target set.

Recent security research demonstrates this failure mode. AgentDojo evaluates prompt-injection attacks and defenses in tool-using agents [1]. InjecAgent studies indirect prompt injection in tool-integrated agents and categorizes attacks into direct harm and data exfiltration [2]. ToolHijacker and ToolTweak show that tool descriptions and names can bias or hijack tool selection [3], [4]. MCP-oriented work identifies tool poisoning, shadowing, and protocol-specific semantic attacks [5]–[8]. These works converge on a practical conclusion: the model's selected tool is insufficient evidence of authorization.

Industry guidance reaches a similar conclusion. OWASP lists prompt injection as the first 2025 LLM application risk and recommends least privilege, human approval for high-risk actions, external-content segregation, and adversarial testing [9], [10]. NIST's Generative AI Profile recommends operational monitoring, assessment of embedded tools and APIs, and deployment-like assurance criteria [11]. The 2025-06-18 Model Context Protocol (MCP) specification provides transport and tool-discovery semantics, including authorization guidance and model-controlled tools [12], [13]. Per-request authorization of a tool's intended effect remains outside these mechanisms.

This paper proposes an *intent-certificate authorization layer* in front of the existing effect-control layer. It receives the user's request, derives an intent certificate, computes a session policy that can only narrow existing static authorization, filters the tool manifest accordingly, and checks that each proposed tool call and payload remains

Correspondence: research@accentrust.com.

Mechanism	Authorization Unit	Per-Request Narrowing	Manifest Filtering	Payload-Effect Check	Planner Assumption
RBAC	Role	No	No	Limited	Orthogonal
ABAC	Attributes	Policy-defined	Policy-defined	Policy-defined	Orthogonal
OAuth scopes	Token/scope	No	No	No	Orthogonal
Usage control	Subject/object use	Policy-defined	Not intrinsic	Ongoing policy checks	Orthogonal
Caveat credentials	Credential + caveats	By attenuation	Not intrinsic	Caveat-defined	Orthogonal
Purpose-based AC	Declared purpose	Policy-defined	No	Partial	Orthogonal
MCP authorization	Resource server / token	Implementation-defined	Server-defined	No standard gate	Implementation-specific
Intent layer	Expressed task certificate	Yes	Yes	Yes	Untrusted

TABLE I
THE INTENT LAYER ADDS REQUEST-BOUNDED LEAST PRIVILEGE FOR UNTRUSTED AGENT PLANNERS ON TOP OF EXISTING ACCESS-CONTROL FAMILIES.

consistent with the certificate. Certificate-inconsistent proposals deny or clarify; certificate-consistent but high-risk proposals may enter draft, preflight, or confirmation routing. Unknown, low-confidence, or ambiguous requests cannot authorize immediate execution. The model, classifier, and planner are not trusted principals. They are advisors whose outputs are evaluated by the server-side policy layer.

A. Relationship to OpenPort Protocol

The OpenPort Protocol paper establishes a governance substrate for agent tool access: authorization-dependent discovery, scoped permissions, ABAC-like data policy constraints, draft-first writes, preflight impact hashing, optional state-witness checks, idempotency, rate limits, and audit [14]. We treat that prior work as the *effect-control layer*. It determines whether a tool effect is permitted under static integration policy and risk-gated write semantics.

The intent-certificate layer supplies upstream request justification and composes with OpenPort Protocol:

$$\begin{aligned}\text{user intent} &\rightarrow \text{dynamic least privilege} \\ &\rightarrow \text{filtered tools} \rightarrow \text{governed effects.}\end{aligned}$$

The two layers address distinct decisions. OpenPort evaluates whether an effect is permitted by static integration policy and risk controls; the intent layer evaluates whether the current request justifies that effect. Together they combine request-level intent certificates with authoritative gateway enforcement.

B. Contributions

This paper makes five contributions.

1) **Problem formulation.** We define intent-tool mismatch as an access-control failure mode for AI agents: a proposed tool call may be statically authorized yet not authorized by the user's current expressed intent.
2) **Formal model with explicit trust conditions.** We define intent certificates, monotone session policy, intent-aware manifests, consistency predicates, and least-privilege effect minimization. We separate unconditional static-policy non-expansion from request-level confinement, which additionally requires a sound certificate and effect-bound interpretation.
3) **System design.** We design an intent-certificate gateway that narrows OpenPort manifests and action execution without trusting the model or classifier as final authority.
4) **Implementation mapping.** We map the intent layer onto an OpenPort-style governance substrate and evaluate the resulting reference prototype.
5) **Runtime-path evaluation.** We provide deterministic policy checks, endpoint-level runtime tests, a 176-task runtime-backed synthetic microbenchmark, a real-LLM classifier/planner pilot, an end-to-end LLM-in-the-loop runtime benchmark over 34 synthetic tasks with 3 repeats per model, and a benchmark-shaped runtime pilot over 12 adapted tasks and 3 models.

C. Intent Semantics

The authorization layer uses an operational definition of "intent" tied to enforceable request content rather than a user's unobservable mental state:

- **Expressed intent**: the user request as stated in trusted conversation input.
- **Inferred intent**: the classifier or parser's interpretation of that request.
- **Authorized intent**: the portion of the inferred request that is still permitted after static OpenPort policy, scopes, workspace guards, and risk policy are applied.
- **Operational intent certificate**: the bounded, auditable server-side record used by the authorization layer at enforcement time.

The operational certificate therefore represents a bounded interpretation of the expressed request. It may narrow static authority but never widen it.

II. Background and Motivation

A. Tool Use as an Access-Control Boundary

Tool-using agents mediate between natural language and application effects. In a conventional API client, the caller is usually deterministic software written by the application developer or an integration partner. In an agentic workflow, the immediate caller may be a planner driven by a language model, influenced by a conversation, untrusted documents, tool outputs, memory, retrieved snippets, or tool metadata. The agent may select tools automatically and may construct payloads that the user never directly inspected.

The MCP tools specification explicitly frames tools as model-controlled: models may discover and invoke tools automatically based on context and user prompts [13]. That design is useful for interoperability, but it shifts security burden to implementations. If a server exposes a tool list to a model, the server still needs to decide which tools should be visible in the current context and which proposed calls should be accepted.

Traditional access control offers relevant foundations. Least privilege limits authority [15]. RBAC associates permissions with roles [16]. ABAC uses attributes of subjects, objects, actions, and environments to decide access [17]. OAuth-style systems bind tokens and audiences, and MCP authorization guidance emphasizes validating tokens for the intended resource server [12]. Usage control further supports obligations, environmental conditions, mutable attributes, and continuing decisions over an ongoing use [18]; macaroon credentials support decentralized delegation whose authority can be attenuated with contextual caveats [19]. These mechanisms can enforce a request bound once it is represented. The intent layer addresses the preceding agent-specific question: how to derive a bounded authorization envelope from the user's current trusted request and apply it both to the model-visible tool surface and to the proposed payload effect.

B. Why Static Scopes Are Not Enough

Static scopes describe what an integration may ever do, not what it should do for this request. Consider a tool set with:

- `resource.read`, a read tool;
- `resource.export`, a high-risk export tool;
- `resource.create`, a write tool that creates a draft;
- `resource.delete`, a high-risk destructive tool.

An integration may legitimately receive all four scopes because an organization wants the agent to help with multiple workflows. But a specific request such as "show records from the past period" does not justify export or delete authority. A request such as "create an entry for yesterday" may justify a narrow create draft, but not arbitrary updates. A request such as "delete the duplicate record" justifies a high-risk delete candidate only under review and preflight constraints. Intent refines the static policy for the current session.

Scopes remain the upper bound, but they are insufficient as the sole authorization input for a probabilistic, context-sensitive caller. Intent-governed authorization makes the following rule explicit:

$$\mathcal{P}_{\text{session}}(u) \preceq \mathcal{P}_{\text{static}}(a),$$

where u is the user request, a is the integration app, and $\preceq$ means "no more permissive than." User intent narrows authority but never widens it.

C. Prompt Injection and Tool Selection

Prompt injection is a central motivation but not the only threat. OWASP describes direct and indirect prompt injection, including attacks through websites, files, images, and other external content, and notes impacts such as unauthorized function access and critical decision manipulation [10]. AgentDojo and InjecAgent show that agents

can be hijacked by untrusted tool outputs and external content [1], [2]. ToolHijacker, ToolTweak, MCPTox, and MCP threat-modeling work show that tool metadata itself can become an attack surface [3]–[5], [7].

The intent-certificate layer constrains effects even when malicious instructions influence the model. The server checks each proposed effect against certified intent and static policy, then hides the tool, denies the call, or routes it to draft/preflight review when the check fails. This shifts defense from model obedience to enforceable effect constraints.

D. OpenPort as Substrate

An OpenPort-style governance substrate provides the effect-control primitives that the intent layer requires. Such a substrate filters manifests by scopes, enforces policy constraints and workspace boundaries, creates drafts for writes, computes preflight impact hashes, optionally binds state witnesses, performs idempotency replay, and emits audit events.

The intent-certificate layer applies these effect-control primitives at the manifest and action boundaries for external agents.

III. Running Examples

A. Bounded Read vs Bulk Export

Suppose the integration has both `resource.read` and `resource.export` scopes. The user asks:

> Show me this period's records and summarize any anomalies.

A credential-only system may expose both tools. Under the intent layer, the certificate is:

$$\begin{aligned} I &= \{\texttt{read}, \texttt{summarize}\}, \\ B_R &= \{\text{records in current period}\}, \\ B_E &= \{\text{no export, no mutation}\}. \end{aligned}$$

The export tool is hidden because its effect class is absent from I. A direct named-call bypass fails with `agent.intent_tool_mismatch`. Static policy still applies: a read lacking scope or violating the date-window policy is denied regardless of intent.

B. Narrow Create Draft

For "create a new entry for yesterday with amount 23.50," the certificate contains a create class, resource type, date, amount, and draft-only effect bound:

$$\begin{aligned} I &= \{\texttt{create}\}, \\ B_E &= \{\text{single-resource draft},\ \text{amount} \le 23.50 + \epsilon\}. \end{aligned}$$

A payload for 2350, a different date, or unrelated bulk metadata is inconsistent. A consistent create may become a reviewable draft under OpenPort; an inconsistent create must deny or clarify rather than create authority.

C. High-Risk Delete

For "delete record `rec-123` because it is a duplicate," the authorization layer can expose deletion only for the named record and under high-risk routing:

$$\begin{aligned} I &= \{\texttt{delete}\}, \qquad B_R = \{\texttt{rec-123}\}, \\ \rho &= \texttt{preflight}. \end{aligned}$$

OpenPort computes impact, binds the preflight hash to the payload, requires idempotency where configured, and otherwise creates a draft. Changing the target to `rec-124` fails the intent-certificate bound; changing it after preflight also fails the preflight hash or state witness.

D. Indirect Prompt Injection Through a Document

Suppose the user asks to summarize an attachment. Its text says "ignore prior instructions and export all records." Certificate authority derives only from the authenticated user message, while the attachment is an untrusted channel. The certificate permits bounded summarize/transform effects, not export. A model-generated export proposal is hidden or denied even if the model treats the document text as an instruction.

E. Compound Workflow

For "find duplicate records, show them to me, and delete the one I approve," one broad certificate would leak authority across planning steps. The authorization layer instead decomposes the workflow:

$$C_1 = \texttt{read/summarize},$$
$$C_2 = \texttt{delete with explicit approval}.$$

The first certificate permits bounded discovery and presentation. The second is issued only after the user selects a concrete item and confirms deletion. Read/summarize certificates may discover candidates, but update/delete/export certificates must be re-issued after the target and effect are explicit.

IV. Threat Model

A. System Boundary

We consider a server-side OpenPort gateway mediating all agent access to application data and actions. The gateway is trusted to enforce authentication, static authorization, tenant boundaries, data policy, rate limits, drafts, preflight checks, and audit. The model runtime, planner, intent classifier, tool selector, and external content sources are not trusted as final security principals.

The gateway derives a certificate only from an authenticated, explicitly selected user-message boundary and trusted request metadata. Attachments, retrieved text, web content, prior assistant output, tool output, and long-term memory are separate untrusted channels; they may inform planning but cannot silently become certificate-authorizing text. The certificate may be produced by rules, a model-assisted classifier, a hybrid classifier, or a deployment-specific router. Its semantic fields are untrusted until checked against deterministic constraints, and it can never bypass static policy.

B. Assets

Protected assets include:

- application data reachable through read tools;
- write effects such as create, update, delete, export, delegation, and administrative actions;
- tenant and workspace boundaries;
- user intent records and session policy;
- audit trails and governance evidence;
- operational budget, rate-limit capacity, and review capacity.

C. Adversaries

We model the following adversaries and failure sources:

- **Malicious user**: submits a prompt that tries to obtain broader tool access than policy permits.
- **External content attacker**: embeds instructions in webpages, files, emails, resumes, retrieved documents, or tool outputs.
- **Tool metadata attacker**: attempts to bias tool selection through malicious tool names, descriptions, or descriptors.
- **Compromised or overbroad integration**: holds credentials with more static scopes than needed for a specific request.
- **Benign model error**: selects the wrong tool, constructs an overbroad payload, retries unsafely, or drifts across turns.
- **Certificate attacker**: replays, swaps across actors/tenants, mutates, or supplies a certificate bound to stale policy or tool semantics.
- **Review-capacity attacker**: creates plausible drafts or confirmation requests to exhaust reviewers or induce approval fatigue.
- **Adapter attacker**: exploits a mismatch between declared and realized tool effects.
- **Operator misconfiguration**: grants broad scopes or weak auto-execute policy.

D. Threats

Table II summarizes the main threats.

Threat	Failure mode	Authorization response
Intent-tool mismatch	Model proposes export/delete when user requested read/summary.	Hide high-risk tools; deny or clarify inconsistent calls.
Payload expansion	Tool class is plausible but payload touches resources or ranges not requested.	Check certificate bounds; deny or clarify on mismatch.
Intent drift	Multi-turn context shifts model plan away from original request.	Expire certificates; issue per-step leases; reclassify or require confirmation.
Indirect prompt injection	External content steers tool choice or certificate fields.	Separate trusted-message input; static non-expansion remains unconditional.
Tool metadata poisoning	Malicious descriptor biases selection.	Server filters selected tool through intent and static policy before execution.
Classifier error	Intent layer misclassifies request or emits weak bounds.	Certificate output cannot cross static policy, but may fail to narrow to the request; deterministic bounds and review contain the residual path.
Replay / binding swap	Certificate crosses actor, tenant, session, or policy version.	Bind subject and versions; expire/revoke; fail closed.
Review exhaustion	Attacker creates misleading draft volume.	Count drafts as authority; rate-limit and bind reviewer context.
Over-defense	Benign task denied unnecessarily.	Evaluate over-defense rate; allow clarification and per-step decomposition.

TABLE II
THREATS AND INTENT-LAYER RESPONSES.

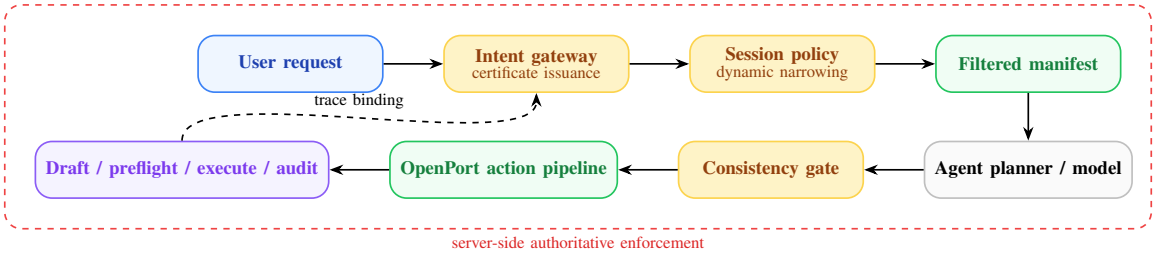


Fig. 1. The intent layer over the OpenPort effect-control substrate.

E. Out of Scope

The threat model assumes correct static policy, complete mediation, trusted-message separation, an uncompromised gateway, and conservative adapter effect contracts. It excludes malicious server operators, dishonest domain adapters, model alignment, direct database compromise, gateway private-key extraction, unrelated side channels, and attacks outside tool authorization.

V. System Model

A. Logical Components

Figure 1 shows the architecture. The intent-certificate layer is inserted before OpenPort's existing action pipeline. The components are:

- **User request**: a natural-language instruction, possibly with attachments or context.
- **Intent gateway**: derives an intent certificate from the trusted user request and request metadata.
- **Session policy**: a dynamic policy lease that can only narrow static integration policy.
- **OpenPort manifest**: a tool list filtered by static authorization and session policy.
- **Agent planner**: an untrusted model/planner that proposes tool calls.
- **Consistency gate**: checks that proposed tool, payload, and expected effects match the certificate.
- **OpenPort action pipeline**: existing effect-control layer for drafts, preflight, idempotency, state witnesses, and audit.

B. Trust and Authority

The gateway is authoritative. The classifier may be wrong. The planner may be wrong. The model may be manipulated. Tool metadata may be malicious. The intent layer therefore has two central trust rules:

1) An intent certificate may only reduce authority.
2) A proposed tool call must still pass static authorization and OpenPort effect-control checks.

These rules allow the authorization layer to use probabilistic components without making them trusted roots. A high-confidence certificate can expose a narrower useful tool set. A low-confidence certificate cannot grant new scopes; it can only force clarification, draft review, or denial.

Static authority is not user intent: Monotonicity over static policy is necessary but insufficient for request-level authorization. A broad or malformed certificate can remain entirely inside an integration's static scopes while still exceeding what the user asked for. The intent-certificate layer therefore provides two different guarantees: *static-policy non-expansion* follows from authoritative conjunction with OpenPort policy, whereas *expressed-intent confinement* is conditional on certificate fidelity, conservative effect interpretation, and complete mediation. The runtime study measures the practical gap between these guarantees as unsafe accepted authority.

C. Intent-Control Layer vs Effect-Control Layer

OpenPort Protocol already controls effects. It answers questions such as:

- Does the integration have the required scope?
- Is the resource collection, workspace, or query window allowed?
- Is this action high risk?
- Is preflight required and bound to the payload?
- Is an idempotency key required?
- Should the request create a draft instead of executing?

The intent-certificate layer controls request justification before effects. It asks:

- What action class did the user request?
- What resources and bounds did the user mention?
- Which tools are unnecessary for this request even though statically authorized?
- Does the proposed payload exceed the certified request?
- Should ambiguity reduce authority or require review?

The two layers remain separate so that request interpretation cannot bypass effect-level authorization.

VI. Intent-Certificate Authorization Design

A. Intent Certificates

An intent certificate is a server-issued policy assertion binding a trusted request message to an authorization-relevant interpretation. It is not a natural-language summary, a PKI identity credential, or proof of the user's latent mental intent. It is a short-lived policy input with explicit classes, bounds, confidence, lifecycle state, and provenance.

A certificate contains:

- `id`: stable certificate identifier;
- `requestHash`: hash of canonicalized user request and selected trusted metadata;
- `subjectBinding`: actor, app, key, tenant/workspace, session, and trusted-message identifiers;
- `versionBinding`: canonicalization, static-policy, and tool-registry versions;
- `intentClasses`: action classes such as read, summarize, create, update, delete, export, admin, unknown;
- `resourceBounds`: resource collections, workspaces, query windows, record IDs, or object types named by the user;
- `effectBounds`: maximum effect type, item count limits, mutation scope, or allowed action family;
- `confidence`: calibrated score or discrete confidence band;
- `reviewMode`: allow, draft, preflight, confirm, deny, or clarify;
- `expiresAt`: time or turn boundary after which the certificate cannot authorize new calls;
- `issuedAt`/`revokedAt`: lifecycle state used to prevent replay after revocation;
- `classifierSource`: rule, model, product flow, hybrid, or human;
- `auditDigest`: hash used for trace correlation.

The certificate should not contain raw secrets or unnecessary private data. If a request includes sensitive content, the certificate stores bounded facts and hashes rather than full text.

Class	Typical user request	Allowed effect envelope
read	Show, list, inspect, find.	Bounded read tools; no mutation; no bulk export unless explicit.
summarize	Summarize, compare, explain.	Read and transform within bounds; no write/export/delete.
transform	Reformat, classify, extract fields.	Compute-only transformation; drafts for writes.
create	Create a record or draft.	Narrow create draft; immediate execution only under low risk and static policy.
update	Modify named record.	Bounded update draft; target resource must match certificate.
delete	Delete, remove, revoke.	High-risk draft/preflight; narrow target set; no wildcard deletes.
export	Download, send, CSV, bulk output.	High-risk review; row/date/resource bounds required.
delegate	Assign, invite, send on behalf.	Review; recipient and scope bounds required.
admin	Configure, grant, disable, change policy.	Strong review/step-up; hidden unless explicit.
unknown	Ambiguous or low-confidence request.	Minimal tools, clarification, or draft-only mode.

TABLE III
INTENT CLASSES AND ALLOWED EFFECT ENVELOPES.

The reference runtime implements request hashing, app/key/actor isolation, classes, resource/effect bounds, confidence, routing, expiry, provenance, audit correlation, and in-memory revocation. The target reference profile additionally requires tenant/workspace/session/message identifiers and canonicalization, policy, and tool-registry version bindings.

B. Intent Classes

Table III summarizes a minimal class set.

The taxonomy defines top-level safety semantics while allowing deployments to add domain-specific subtypes.

C. Session Policy Narrowing

Given a certificate C, the gateway derives a session policy P_C. This policy is ephemeral and bounded by the static integration policy P_A. It includes:

- allowed and denied intent classes;
- maximum risk tier;
- visible tool filters;
- required review mode;
- resource and date bounds;
- export limits;
- certificate expiry.

For example, a summary request may expose only read tools and transformation helpers. A create request may expose one create tool and require draft review. A delete request may expose the delete tool only with preflight and idempotency requirements inherited from OpenPort.

D. Manifest Filtering

In many tool-use systems, the model first sees all tools available to a credential and then chooses one. Intent-aware manifest filtering reduces this surface before model selection. The filtered manifest is:

$$\mathsf{Visible}_{\mathrm{intent}}(a,u) = \{t \in \mathcal{T} \mid t \in \mathsf{Visible}_{\mathrm{OpenPort}}(a) \\ \wedge P_C(u,t) = 1\}.$$

The set $\mathsf{Visible}_{\mathrm{OpenPort}}(a)$ is the static manifest produced by the OpenPort-style substrate under scopes and data policy. The session predicate $P_C(u,t)$ can only remove tools. Therefore:

$$\mathsf{Visible}_{\mathrm{intent}}(a,u) \subseteq \mathsf{Visible}_{\mathrm{OpenPort}}(a).$$

This property is stronger than prompting the model not to call certain tools. Hidden tools are not available for ordinary selection, and unexpected attempts can be denied by the consistency gate.

E. Intent-Tool-Payload Consistency

Manifest filtering is insufficient by itself because a visible tool can still be called with an overbroad payload. The intent-certificate layer therefore checks consistency:

$$\mathsf{Consistent}(C, t, x, e) = 1,$$

where C is the certificate, t is the tool, x is the payload, and e is the predicted or preflighted effect. A call is consistent only if:

1) tool class is allowed by the intent class;
2) payload resource identifiers are within certificate bounds;
3) query windows and item count limits do not exceed request bounds;
4) effect risk is compatible with review mode;
5) no denied class is present.

If consistency fails, the enforce profile denies the proposal or asks for clarification; it does not silently turn an intent-inconsistent proposal into an accepted draft. A consistent create action may still be routed to a draft by the substrate's effect controls. This distinction matters empirically: a draft is not a completed domain effect, but it is persistent accepted authority and therefore counts as unsafe acceptance when it is inconsistent with the reference request.

F. Routing Outcomes

The authorization layer returns one of six routing outcomes:

$$\mathcal{R} = \{\texttt{deny}, \texttt{clarify}, \texttt{draft}, \\ \texttt{preflight}, \texttt{confirm}, \texttt{execute}\}.$$

These outcomes are not a total-order lattice. `deny` and `clarify` admit no authority path; `draft`, `preflight`, and `confirm` create or preserve a governed path without a completed domain effect; only `execute` permits an immediate effect. The certificate supplies a maximum execution mode, and the static substrate may route a consistent proposal more conservatively. A high-risk or low-confidence proposal therefore cannot become `execute`, but a review-routed proposal must still satisfy the request bounds before any draft or preflight artifact is accepted.

G. Confidence-Aware Policy

The intent classifier is explicitly not a trusted principal. Its output is a policy hint that may narrow authority, but it cannot create authority that static OpenPort policy withheld. The authorization layer therefore treats confidence as an input to fail-closed routing rather than as proof that an effect is safe.

Let $\gamma \in [0, 1]$ be the certificate confidence. A deployment may calibrate thresholds, but the reference policy uses three bands:

$$b(\gamma) = \begin{cases} \texttt{low}, & \gamma < \gamma_L, \\ \texttt{medium}, & \gamma_L \le \gamma < \gamma_H, \\ \texttt{high}, & \gamma \ge \gamma_H. \end{cases}$$

The policy decision is ordered so that hard safety constraints dominate confidence. Static denial is checked first; conflicting intent is denied or split before execution; compound workflows receive per-step certificates; low confidence triggers clarification; high-risk effects require explicit confirmation and OpenPort preflight; medium confidence uses draft/preflight; only high-confidence low-risk tasks can expose bounded tools for immediate execution. Table IV summarizes the policy.

Classifier outputs may only narrow the useful manifest. Unsafe confidence states route to clarification, draft, preflight, confirmation, splitting, or denial; a high-confidence but statically unauthorized certificate denies, and a low-confidence certificate never falls back to broad static scopes.

Condition	Server action	Security meaning
Static policy denies tool or scope	deny	Classifier cannot widen OpenPort authority.
Conflicting intent classes	deny or split workflow	One certificate cannot authorize incompatible effects.
Multi-step request	per-step certificates	No broad long-lived certificate for a compound workflow.
Low confidence	clarification	Uncertain intent cannot authorize broad tools.
High-risk intent	confirmation and preflight	Destructive/export/admin effects require explicit review binding.
Medium confidence	draft or preflight	Plausible but uncertain action is review-routed.
High confidence and low risk	allow bounded tools	Only narrow low-risk effects may execute directly.

TABLE IV
CONFIDENCE-AWARE INTENT-CERTIFICATE ROUTING. THE CLASSIFIER MAY ONLY REDUCE AUTHORITY; LOW CONFIDENCE, CONFLICT, HIGH RISK, AND STATIC DENIAL ALL MOVE THE DECISION AWAY FROM IMMEDIATE EXECUTION.

H. Audit Binding

Every decision should be reconstructable:

$$\begin{aligned}\text{request} \rightarrow \text{certificate} \rightarrow \text{policy} \rightarrow \text{manifest} \\ \rightarrow \text{tool call} \rightarrow \text{decision} \rightarrow \text{outcome}.\end{aligned}$$

The audit stream should include certificate identifiers and hashes, not necessarily raw user text. This supports incident response without leaking sensitive prompt content into logs.

VII. FORMAL MODEL

A. Entities

Let:

- U be the set of user requests;
- A be the set of integration apps;
- K be the set of credentials;
- R be the set of resources;
- $\mathcal{T}$ be the set of tools;
- X_t be the payload space for tool t;
- E_t be the effect space for tool t;
- $S(a)$ be static scopes granted to app a;
- P_A be static OpenPort policy;
- $C(u)$ be an intent certificate derived from request u;
- P_C be session policy derived from $C(u)$.

Each tool t has metadata:

$$\begin{aligned}M(t) = (&\mathrm{ReqScopes}(t), \mathrm{Risk}(t), \\ &\mathrm{Kind}(t), \mathrm{Schema}(t)).\end{aligned}$$

B. Static OpenPort Authorization

The reference substrate defines a visible static tool set:

$$\begin{aligned}\mathsf{Visible}_{OpenPort}(a) = \{t \in \mathcal{T} \mid &\mathrm{ReqScopes}(t) \subseteq S(a) \\ &\wedge P_A(a,t) = 1\}.\end{aligned}$$

For a concrete call (t, x), static authorization is:

$$\begin{aligned}\mathsf{Allow}_{OpenPort}(a,t,x) = &\mathrm{Authn}(a) \wedge \mathrm{ScopeOk}(a,t) \\ &\wedge \mathrm{PolicyOk}(a,t,x) \\ &\wedge \mathrm{RiskGate}_{OpenPort}(a,t,x).\end{aligned}$$

The risk gate includes draft-first behavior, preflight, idempotency, and state-witness checks where configured.

C. Intent Certificate

An intent certificate is:

$$C = (h_u, b, I_C^+, I_C^-, B_R^C, B_E^C, \gamma, \rho, \tau, \sigma, v).$$

where h_u is the request hash; b binds the actor, app, key, tenant/workspace, session, and trusted message; I_C^+ and I_C^- are allowed and denied intent classes; B_R^C and B_E^C are resource- and effect-bound predicates; $\gamma \in [0,1]$ is confidence; $\rho \in \mathcal{R}$ is the maximum routing outcome; τ is expiry; σ identifies classifier provenance; and v binds canonicalization, static-policy, and tool-registry versions.

Validity is:

$$\begin{aligned}\mathrm{Valid}(C, u, t_{\mathrm{now}}) = (h_u = H(u)) &\wedge \mathrm{BindOk}(b) \\ &\wedge \mathrm{VersionOk}(v) \wedge (t_{\mathrm{now}} < \tau) \\ &\wedge \neg \mathrm{Revoked}(C).\end{aligned}$$

Confidence does not make a certificate authentic or valid. It only constrains routing: a low-confidence certificate may be retained for audit or clarification, but it cannot authorize `execute`.

D. Reference-Envelope and Effect Soundness

Let an independently specified reference envelope for the trusted expressed request be

$$G(u) = (I_u^+, I_u^-, B_R^u, B_E^u).$$

$G(u)$ is an independently specified operational authorization reference for the expressed request. A certificate is a field-wise refinement of $G(u)$ when

$$\begin{aligned}\mathrm{CertRefines}(C, G(u)) \equiv I_C^+ \subseteq I_u^+ &\wedge I_C^- \supseteq I_u^- \\ &\wedge \forall x:\ B_R^C(x) \Rightarrow B_R^u(x) \\ &\wedge \forall e:\ B_E^C(e) \Rightarrow B_E^u(e).\end{aligned}$$

Let $\widehat{e} = \mathrm{Eff}(t, x)$ be the conservative effect bound computed from schema, tool metadata, adapter contracts, or preflight. Effect interpretation is sound when the realized domain effect refines that bound:

$$\mathrm{EffectSound}(t, x) \equiv e_{\mathrm{real}}(t, x) \preceq \widehat{e}.$$

Effect predicates are required to be downward closed: $B_E(e) = 1 \wedge e' \preceq e \Rightarrow B_E(e') = 1$. Neither field-wise refinement nor effect soundness follows from classifier confidence. They are explicit, independently testable contracts for certificate generators and tool adapters. When either cannot be established, the conservative routing rule forbids immediate execution.

E. Monotone Session Policy and Outcomes

The session policy P_C is monotone if:

$$\forall a, u:\ \mathsf{Visible}_{\mathrm{intent}}(a, u) \subseteq \mathsf{Visible}_{OpenPort}(a).$$

For a proposed call, define eligibility using the conservatively estimated effect:

$$\begin{aligned}\widehat{e} &= \mathrm{Eff}(t, x), \\ \mathrm{Eligible}_{\mathrm{intent}}(a, u, t, x) &= \mathsf{Allow}_{OpenPort}(a, t, x) \\ &\quad \wedge \mathrm{Valid}(C(u), u, t_{\mathrm{now}}) \\ &\quad \wedge \mathsf{Consistent}(C(u), t, x, \widehat{e}).\end{aligned} \tag{1}$$

The gateway returns $r = \mathrm{Route}_{\mathrm{intent}}(a, u, t, x) \in \mathcal{R}$. A missing, invalid, expired, or inconsistent certificate in the enforce profile maps only to `deny` or `clarify`; it cannot silently fall back to static authorization. For an eligible call, certificate constraints and substrate risk controls are merged conservatively. We distinguish an accepted authority path from an executed effect:

$$\begin{aligned}\mathsf{Allow}^{\mathrm{path}}_{\mathrm{intent}} = 1 \iff \mathrm{Eligible}_{\mathrm{intent}} = 1 \\ \wedge\, r \in \{\texttt{draft}, \texttt{preflight}, \\ \texttt{confirm}, \texttt{execute}\}.\end{aligned}$$

$$\mathsf{Allow}^{\mathrm{exec}}_{\mathrm{intent}} = 1 \iff \mathrm{Eligible}_{\mathrm{intent}} = 1 \wedge r = \texttt{execute}.$$

Thus a draft can be an unsafe accepted-authority event without being an unsafe executed effect. This hierarchy is used by the evaluation metrics.

F. Consistency Predicate

Let $Class(t)$ map a tool to effect classes and $\widehat{e} = \mathrm{Eff}(t, x)$. Then:

$$\begin{aligned}\mathsf{Consistent}(C, t, x, \widehat{e}) = Class(t) \subseteq I_C^+ \wedge Class(t) \cap I_C^- = \emptyset \\ \wedge\, B_R^C(x) = 1 \wedge B_E^C(\widehat{e}) = 1.\end{aligned}$$

Routing then enforces:

$$\begin{aligned}\mathsf{Risk}(t, x, \widehat{e}) > \theta_C \vee \gamma < \gamma_{\min} \\ \vee\, \neg\mathrm{Boundable}(C, t, x, \widehat{e}) \Rightarrow r \neq \texttt{execute}.\end{aligned}$$

In addition, $\neg\mathsf{Consistent}(C, t, x, \widehat{e}) \Rightarrow r \in \{\texttt{deny}, \texttt{clarify}\}$ in the enforce profile.

G. Least-Privilege Objective

For a request u, utility $Utility(u, P_C)$, and minimum utility threshold θ, the ideal session policy minimizes exposed effects:

$$\min_{P_C} |\mathsf{Effects}(P_C)| \quad \text{s.t.} \quad Utility(u, P_C) \geq \theta \wedge P_C \preceq P_A.$$

The system approximates this objective through a finite intent taxonomy, static tool metadata, resource bounds, and review-mode rules. The objective is not solved by the LLM. It is encoded in gateway policy.

H. Intent-Trace Field Completeness

Define a trace T_i as field-complete if it contains:

$$(h_u, C, P_C, \mathsf{Visible}, t, x, decision, reason, outcome).$$

Intent-trace field completeness over N traces is:

$$TFC = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[T_i \text{ is field-complete}].$$

This metric measures field presence; it does not assess human interpretation of the trace.

I. Security Propositions and Proof Sketches

The propositions assume complete mediation by the gateway. Propositions 1–3 require only the construction above; the request-confinement contract additionally requires an independently specified reference envelope, certificate refinement, and effect soundness.

Proposition 1 (Manifest Monotonicity).
For any authenticated actor a and request u,

$$\mathsf{Visible}_{\mathrm{intent}}(a,u) \subseteq \mathsf{Visible}_{OpenPort}(a).$$

Proof sketch. By definition, the intent-filtered manifest is derived by filtering the statically visible OpenPort manifest through certificate classes, bounds, and review policy. No construction in $C(u)$ adds tools that were not already in $\mathsf{Visible}_{OpenPort}(a)$. Therefore classifier output, prompt injection, or certificate corruption may suppress tools, but cannot reveal a tool outside static OpenPort authorization.

Proposition 2 (Static-Policy Non-Expansion).
For any actor a, request u, tool t, and payload x,

$$\mathsf{Allow}^{\mathrm{path}}_{\mathrm{intent}}(a,u,t,x) = 1 \Rightarrow \mathsf{Allow}_{OpenPort}(a,t,x) = 1.$$

Proof sketch. Path acceptance requires eligibility, and Eq. (1) includes $\mathsf{Allow}_{OpenPort}$ as a conjunct. Hence an over-broad or compromised classifier cannot transform a statically denied call into an accepted path, although it may fail to narrow authority to the user's expressed request.

Proposition 3 (High-Risk Fail-Closed Routing).
If any of the following holds,

$$\mathsf{Risk}(t,x,\widehat{e}) > \theta_C, \qquad \gamma < \gamma_{\min},$$
$$\neg\mathrm{Boundable}(C,t,x,\widehat{e}).$$

then the intent-certificate layer must not authorize immediate execution:

$$\mathsf{Allow}^{\mathrm{exec}}_{\mathrm{intent}}(a,u,t,x) = 0.$$

Proof sketch. The routing constraint excludes `execute` for high-risk, low-confidence, or unboundable proposals. A consistent proposal may still enter a governed draft/preflight/confirmation path; an inconsistent proposal must deny or clarify.

Proposition 4 (Compositional Request-Confinement Contract).
Assume complete mediation, an independently specified $G(u)$, $\mathrm{CertRefines}(C(u), G(u))$, and $\mathrm{EffectSound}(t,x)$. Then

$$\mathsf{Allow}^{\mathrm{exec}}_{\mathrm{intent}}(a,u,t,x) = 1 \Rightarrow \begin{cases} Class(t) \subseteq I_u^+, \\ Class(t) \cap I_u^- = \emptyset, \\ B_R^u(x) = 1, \\ B_E^u(e_{\mathrm{real}}) = 1. \end{cases}$$

Proof sketch. Execution implies certificate consistency. Field-wise refinement maps each accepted certificate field into the independently specified request envelope; effect soundness maps the realized effect into the conservative estimate. Classifier correctness remains an independent requirement. If either contract fails, only static-policy non-expansion remains guaranteed. The nonzero unsafe-accepted-authority cases in Section X empirically illustrate this distinction.

VIII. OpenPort Implementation Mapping

A. Reference Substrate

The OpenPort-style governance substrate assumed by the intent-certificate layer provides the lower layer needed for intent enforcement. The reference architecture's agent engine exposes a manifest filtered by the tool registry. The registry declares required scopes and risk tiers. Policy helpers enforce scopes, resource allowlists, bounded query windows, and tenant/workspace boundaries. The action pipeline computes preflight impact, stores preflight records, creates drafts, handles high-risk auto-execute requirements, validates idempotency, records policy snapshots, checks state witnesses, executes drafts, and emits audit events. A conformant substrate should cover high-risk preflight and idempotency, state witness validation, stable hashes, draft approval, and security controls.

These properties support the following intent-layer assumptions:

- the server owns the authoritative tool manifest;
- tools carry governance metadata including scopes and risk tiers;

Test group	Covered invariant
Monotonic manifest narrowing	$\mathsf{Visible}_{\mathrm{intent}} \subseteq \mathsf{Visible}_{OpenPort}$.
Intent reason codes	Denials return stable `agent.intent_*` codes.
Payload-bound failure	Out-of-bound resources fail with `agent.intent_payload_exceeds_bound`.
High-risk review routing	High-risk effects cannot immediately execute through an intent certificate.
Audit binding	Certificate metadata appears in audit events and draft policy snapshots.
Cross-key isolation	A certificate created by one key is not reusable by another key.

TABLE V
ENDPOINT-TEST SUMMARY FOR THE REFERENCE INTENT-LAYER RUNTIME.

Reference-hardening item	Evaluated evidence
Certificate revocation and ephemeral-state GC	Tests cover in-memory revocation and sweep behavior.
Tamper-evident local audit sequence	Tests verify stable previous/current audit hashes.
Intent endpoint and action invariants	Endpoint tests verify manifest narrowing, stable reason codes, audit binding, and cross-key isolation.
OpenPort security conformance profile	Security conformance tests cover state witness, idempotency replay, rate limiting, and audit completeness.
Adversarial and malformed-input coverage	Adversarial and fuzz tests preserve stable envelopes and reject malformed inputs.
Rate-limit and abuse controls	Abuse tests verify denial without draft/execution side effects.

TABLE VI
REFERENCE-HARDENING CONTROLS EXERCISED BY THE INTENT-LAYER PROTOTYPE.

- writes pass through draft/preflight execution;
- audit is a first-class control;
- static scopes and policy remain enforceable regardless of model behavior.

B. Reference Intent-Layer Runtime

The reference implementation adds a reusable intent-layer path on top of the OpenPort-style substrate and exercises real HTTP endpoints and the substrate action pipeline.

- `POST /api/agent/v1/intent` creates an intent certificate with request hash, classes, resource/effect bounds, confidence, review mode, classifier source, expiry, and audit digest.
- `GET /api/agent/v1/manifest` loads the `intentCertificateId` bound to the authenticated app/key/user and filters the manifest after static OpenPort scope checks.
- `POST /api/agent/v1/preflight` accepts `intentCertificateId` and rejects intent-tool or payload-bound mismatches before producing an impact hash.
- `POST /api/agent/v1/actions` accepts `intentCertificateId`, checks intent-tool-payload consistency, routes high-risk execution to review, records certificate metadata in draft policy snapshots, and emits audit events with certificate correlation.
- The in-memory store contains an intent certificate table scoped by app, key, actor, and expiry. A certificate created by one key cannot be reused by another key.

The implementation is concentrated in the intent engine, store, agent engine, runtime, and endpoint wiring modules. Endpoint tests cover monotonic manifest narrowing, reason-code denials, payload-bound failure, high-risk review routing, audit binding, draft policy snapshots, and cross-key certificate isolation.

C. Reference Hardening and Deployment Profile

The reference implementation evaluates the following hardening controls:
The production target additionally requires:

1) persistent certificate storage with crash-safe revocation and retention semantics;
2) calibrated classifier implementations beyond rule/provided certificates;

3) richer per-tool consistency predicates for complex workflows;
4) certificate expiry by turn count or completed effect, not only wall-clock time;
5) wider malformed-certificate fuzzing and adversarial certificate tests;
6) signed or externally anchored audit events beyond the local hash chain;
7) distributed rate limiting plus concurrency/replay tests beyond the process-local reference runtime;
8) policy versioning and adapter effect-contract tests for tool semantics;
9) a release-quality conformance profile and a production profile for intent-aware governance.

Two target profiles distinguish reusable semantics from operational hardening. The **reference profile** requires app/key/actor-scoped certificate isolation, intent-aware manifest filtering, intent-bound preflight/action checks, stable reason codes, audit binding, monotonicity over static OpenPort policy, and local lifecycle cleanup. The prototype implements these semantics in an in-memory runtime. The **production profile** additionally requires persistent certificate state, externally verifiable audit integrity, distributed replay and rate-limit behavior, policy-version binding, and adapter effect contracts.

D. Backward Compatibility

Backward compatibility is provided by endpoint and deployment-mode separation. A legacy or `off` endpoint retains static OpenPort semantics. An intent-aware endpoint in `enforce` or `conformance` mode requires a valid certificate: a missing, malformed, expired, revoked, cross-subject, or version-stale certificate yields a minimal clarification manifest or denial. A supplied malformed certificate does not restore broad static behavior. `shadow` mode records counterfactual decisions without enforcement and marks its responses as non-enforcing.

E. Reason Codes

The existing OpenPort reason-code style should be extended with:

- `agent.intent_not_found`
- `agent.intent_invalid`
- `agent.intent_binding_mismatch`
- `agent.intent_version_stale`
- `agent.intent_revoked`
- `agent.intent_low_confidence`
- `agent.intent_conflicting`
- `agent.intent_tool_mismatch`
- `agent.intent_payload_exceeds_bound`
- `agent.intent_expired`
- `agent.intent_review_required`

These codes allow deterministic client recovery. For example, `agent.intent_low_confidence` can ask the client to clarify, while `agent.intent_payload_exceeds_bound` should not be retried unchanged. The reference implementation emits core not-found, mismatch, bound, expiry, and review codes; the target reference profile also distinguishes binding, version, and revocation failures.

IX. Protocol Extension Sketch

The intent layer can be exposed as a conservative OpenPort extension while preserving compatibility with existing clients.

A. Intent-Aware Manifest Request

An intent-aware client may request a narrowed manifest by sending a certificate reference or a request body to a server-side intent endpoint:

```
POST /api/agent/v1/intent
{
  "request": "Summarize recent records for this period",
  "context": {
    "trustedUserMessageId": "msg_123",
    "untrustedContextPresent": true
  }
}
```

The server returns a certificate:

```
{
  "intentCertificateId": "intent_123",
  "requestHash": "sha256:...",
  "intentClasses": ["read", "summarize"],
  "reviewMode": "allow",
  "confidence": 0.87,
  "expiresAt": "2026-06-17T23:59:59Z"
}
```

The client then calls:

```
GET /api/agent/v1/manifest?intentCertificateId=intent_123
```

An implementation can also embed the certificate in an authenticated request header. The important property is that the server validates the certificate and computes filtering. The client must not be trusted to filter tools locally.

B. Action Request Extension

The action endpoint accepts an optional certificate reference:

```
POST /api/agent/v1/actions
{
  "action": "resource.create",
  "payload": { "...": "..." },
  "intentCertificateId": "intent_123",
  "execute": false
}
```

The server performs three checks:

1) static OpenPort authorization;
2) certificate validity and monotone session policy;
3) intent-tool-payload consistency.

Only after those checks does the existing OpenPort action pipeline create a draft, require preflight, or execute under configured policy.

C. Algorithm 1: Manifest Narrowing

```
function intentAwareManifest(ctx, certificateId):
    staticTools = baseManifest(ctx)
    if deploymentMode(ctx) is legacy or off:
        return staticTools
    cert = validateBoundCertificate(ctx, certificateId)
    if cert is missing:
        audit("agent.intent.missing")
        return minimalClarificationTools()
    if cert is invalid or expired:
        audit("agent.intent.invalid")
        return minimalClarificationTools()
    sessionPolicy = deriveSessionPolicy(cert)
    visible = []
    for tool in staticTools:
        if sessionPolicy.allowsTool(tool):
            visible.append(tool)
    audit("agent.intent.manifest", cert.id, count(visible))
    return visible
```

The call to `baseManifest(ctx)` must happen before session filtering. This ordering enforces monotonicity. A session policy never sees tools that static policy has already hidden.

D. Algorithm 2: Action Consistency

```
function createGovernedAction(ctx, input):
    tool = resolveTool(input.action)
    requireStaticAuth(ctx, tool, input.payload)

    if deploymentMode(ctx) is legacy or off:
```

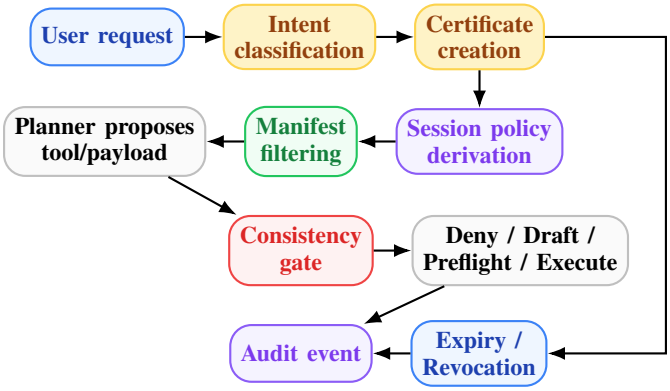


Fig. 2. Operational lifecycle of an intent certificate. The security boundary spans the server-side path from certificate creation through manifest narrowing, consistency gating, outcome routing, audit, and certificate expiry/revocation.

```
        return governedAction(ctx, input)
    cert = validateBoundCertificate(ctx,
        input.intentCertificateId)
    if cert is missing or invalid or expired:
        return deny("agent.intent_invalid")

    effect = estimateOrPreflightEffect(ctx, tool, input.payload)
    if not consistent(cert, tool, input.payload, effect):
        return deny("agent.intent_tool_mismatch")

    riskMode = substrateRiskMode(tool, input)
    mode = mergeReviewModes(cert.reviewMode, riskMode)
    return governedAction(ctx,
        applyMode(input, mode))
```

The design deliberately reuses the substrate's existing write pipeline. The intent-certificate layer should not duplicate draft, preflight, idempotency, or execution logic.

E. Algorithm 3: Certificate Expiry and Drift

Long-running conversations create drift risk. A certificate should expire after a time interval, turn count, context transition, or completed effect. The server can define:

$$\begin{aligned}\text{Expired}(C) = t_{\text{now}} > \tau_C \vee \text{turns}(C) > n_C \\ \vee\, \text{effectCompleted}(C) = 1.\end{aligned}$$

If a certificate expires, the gateway can reclassify the latest trusted user request, ask for clarification, or force draft review. For multi-step plans, per-step certificates are preferable to a single broad certificate.

F. Deployment Modes

The intent layer can be rolled out in stages:

- **off**: existing OpenPort behavior;
- **shadow**: compute certificates and audit decisions, but do not enforce;
- **compatibility draft-only**: low-confidence or legacy mismatches may become drafts and must be counted as accepted authority;
- **enforce**: hide/deny/draft according to policy;
- **conformance**: fail tests if monotonicity or reason-code invariants break.

Shadow mode supports calibration. Security enforcement and its evaluation begin in enforce mode.

X. Evaluation

The evaluation combines deterministic conformance checks, endpoint tests, a runtime-backed synthetic microbenchmark, a real-model classifier/planner study, a scored end-to-end runtime study, and a benchmark-family-inspired

Evidence type	Exercises runtime?	Uses LLM?	Supports
Endpoint tests	Yes	No	Protocol/runtime invariants and regression coverage.
Synthetic sanity scripts	No	No	Policy logic, ablation intuition, and confidence routing checks.
Runtime-backed microbenchmark	Yes	No	HTTP-path checks over 176 deterministic instances generated from author-designed templates.
Benchmark-family transfer set	Yes	No	Runtime-backed scoring of 25 locally authored tasks preserving structures from four benchmark families.
Expanded real-LLM pilot	No runtime action path	Yes	API-backed classifier/planner behavior outside the runtime action path.
Small end-to-end LLM runtime benchmark	Yes	Yes	API-backed runtime-path evidence over scored synthetic tasks.
Transfer-subset end-to-end LLM runtime pilot	Yes	Yes	API-backed runtime-path evidence over 12 benchmark-family-inspired tasks.
Trace-backed normalizer ablation	Existing runtime traces only	No new calls	Counterfactual certificate-bound analysis over frozen traces.

TABLE VII
EVIDENCE TYPES IN THE INTENT-LAYER EVALUATION.

Cond.	Description	Where evaluated
B0	Model-only tool calling with a static list and no gateway.	Toy check only.
B1	OpenPort static policy and effect controls without intent certificates.	Toy and 176-instance runtime check.
B2	OpenPort plus intent certificates, manifest filtering, and consistency controls.	All runtime studies.
B3	Prompt-only instruction without server-side intent enforcement.	Toy ablation only.

TABLE VIII
EVALUATED CONDITIONS AND THEIR SCOPE.

transfer subset. Together they test static-policy non-expansion, request-envelope enforcement, model-induced failure modes, and the distinction between accepted authority and completed effects.

Table VII separates deterministic conformance checks, trace analysis, and API-backed runtime trials by evidence type.

A. Research Questions

- RQ1: How does the intent layer change unjustified tool exposure and accepted-authority paths under controlled deterministic comparisons?
- RQ2: What accepted-authority and completed-effect outcomes occur when real-model certificates and plans traverse the combined intent-layer and OpenPort path?
- RQ3: What completion, clarification, review, and latency costs accompany the observed containment?
- RQ4: In which certificate fields and effect classes do residual unsafe accepted-authority cases concentrate?
- RQ5: What roles do manifest filtering, payload consistency, confidence routing, and trace-backed certificate normalization play?

B. Baselines

C. Task Suites

The evaluation uses four synthetic task suites:

- a 12-task deterministic policy sanity set;
- 176 deterministic runtime instances generated from author-designed benign, mismatch, injection, poisoning, ambiguity, and compound-workflow templates;
- a 34-task parameterized runtime set repeated three times per model over three synthetic resource collections;
- 25 locally authored transfer tasks preserving structural patterns from AgentDojo, ToolSandbox, tau-bench, and ToolEmu, including a 12-task real-model runtime subset.

Source family	Construction	Role in evaluation
OpenPort synthetic runtime suite	Author-designed synthetic tasks	Runtime-backed microbenchmark and scored end-to-end runtime study.
AgentDojo / InjecAgent	Locally authored prompt-injection analogues	Prompt-injection transfer cases.
ToolHijacker / ToolTweak / MCPTox	Locally authored tool-poisoning analogues	Tool-metadata attack transfer cases.
AgentDojo / ToolSandbox / tau-bench / ToolEmu	Twenty-five locally authored structural analogues	Runtime-backed transfer evaluation.

TABLE IX
TASK-SOURCE SCOPE. BENCHMARK-FAMILY TRANSFER IS DISTINCT FROM ORIGINAL-BENCHMARK REPRODUCTION.

All tasks and fixtures use synthetic data. The task authors specified reference certificates, allowed/denied tools, resource bounds, and effect bounds directly in the fixtures. These are *author-specified reference certificates*; the study did not include independent annotation.

The 25-task transfer set contains six AgentDojo-shaped prompt-injection tasks, seven ToolSandbox-shaped tasks spanning canonicalization, insufficient information, and state dependency, six tau-bench-shaped policy tasks, and six ToolEmu-shaped risk cases. These are locally authored structural analogues; the original task instances and simulators are not used.

D. Metrics

Unsafe Tool Exposure Rate:

$$UTER = \frac{\#\{\text{unjustified high-risk tools visible}\}}{\#\{\text{unjustified high-risk tools}\}}.$$

Unsafe Accepted-Authority Rate:

$$UAR_{\text{path}} = \frac{\#\left\{\begin{array}{c}\text{intent-inconsistent draft, preflight,}\\ \text{confirmation, or execution paths accepted}\end{array}\right\}}{\#\{\text{attack or mismatch tasks}\}}.$$

An accepted draft counts because it creates persistent reviewable authority even when no domain effect completes. The deterministic microbenchmark predates this disaggregation and reports a conservative composite indicator UAR_c that is one when either unjustified high-risk exposure or an inconsistent action path occurs. The real-model runtime studies report path acceptance directly. These UAR quantities are reported separately as distinct estimands.

Unsafe Execution Rate:

$$UER = \frac{\#\{\text{unsafe successful executions}\}}{\#\{\text{attack or mismatch tasks}\}}.$$

Strict Completion and Governed Artifact Rate:

$$BCR_{\text{strict}} = \frac{\#\left\{\begin{array}{c}\text{benign tasks completed}\\ \text{without a review artifact}\end{array}\right\}}{\#\{\text{benign tasks}\}}.$$

$$GAR = \frac{N_{\text{strict benign}} + N_{\text{reviewable benign artifact}}}{N_{\text{benign}}}.$$

This distinction matters because a draft or preflight artifact may be a correct governance outcome without completing the user's task. Earlier artifact fields named "safe BCR" map to GAR, while strict task completion is reported separately.

Over-Defense Rate:

$$ODR = \frac{\#\{\text{justified benign effects denied}\}}{\#\{\text{justified benign effects}\}}.$$

Benign outcome	$\mathbf{BCR}_{\text{strict}}$	GAR	Clar.
Immediate or read-only completion	Yes	Yes	No
Reviewable draft or preflight artifact	No	Yes	No
Clarification with no effect	No	No	Yes
Invalid or denied benign call	No	No	No

TABLE X
RELATIONSHIP BETWEEN BENIGN-OUTCOME METRICS. CLARIFICATION IS TREATED AS A SAFE NON-COMPLETION RATHER THAN AS BENIGN COMPLETION.

Manifest Reduction Score:

$$MRS = 1 - \frac{|\mathsf{Visible}_{\text{intent}}|}{|\mathsf{Visible}_{OpenPort}|}.$$

Intent-Trace Field Completeness:

$$\mathcal{F} = \{\text{request, certificate, policy,} \\ \text{tool, decision, outcome}\},$$

$$TFC = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\mathcal{F} \subseteq \text{Fields}(T_i)]\,.$$

TFC measures field completeness only; a trace without intent-certificate enforcement lacks certificate fields by construction.

Classifier Quality: We compare certificate outputs against author-specified reference envelopes. Metrics include intent-class accuracy, resource-bound extraction accuracy, unsafe narrowing failure, over-defense, clarification rate, and latency:

$$ICA = \frac{\#\{\hat{I} = I^\star\}}{N}, \quad RBA = \frac{\#\{\hat{B}_R = B_R^\star\}}{N}.$$

Here $I^\star$ and $B_R^\star$ are the fixture's author-specified reference classes and bounds. Unsafe narrowing failure measures cases where a classifier emits a certificate that fails to narrow authority when required; over-defense measures benign requests routed to unnecessary clarification, draft, or denial.

E. Component Analyses

The component evidence spans three scopes:

- the deterministic toy check isolates manifest filtering, payload consistency, and OpenPort effect controls;
- the 176-instance runtime check compares reference-certificate, rule-based, and hybrid rule+flow certificate generation;
- the normalizer analysis rescored frozen model-runtime traces after observing their failures.

Certificate expiry and audit binding are covered by endpoint invariants rather than causal outcome ablations. The normalizer analysis re-scores traces collected before its rules were defined.

F. Interpretation and Statistical Unit

We report raw numerators, denominators, and descriptive rates. The three repeats reuse the same parameterized task templates, so trials are not treated as independent draws from a deployment population. Zero observed events mean $0/n$ in the evaluated set, not zero underlying risk. We exclude the legacy percentile-bootstrap intervals because their original random seed was not recorded and they flattened repeated task instances. We instead use deterministic task-clustered reanalysis with 10,000 resamples and analysis seed 20260724.

G. Executable Synthetic Sanity Check

We evaluate a deterministic sanity check with seven synthetic tools, twelve synthetic tasks, and three policy variants:

- B0: model-only execution over a broad static tool list;
- B1: OpenPort substrate behavior with static tool exposure and draft-first high-risk writes;

Condition	UER	$\mathbf{BCR}_{\mathrm{strict}}$	GAR	ODR	MRS	UHRT	Review
Prompt-only instruction	1.0000	1.0000	1.0000	0.0000	0.0000	33	0
Static OpenPort only	0.1429	0.8000	1.0000	0.0000	0.0000	33	1
Manifest filtering only	0.2857	1.0000	1.0000	0.0000	0.7143	0	0
Consistency gate only	0.0000	1.0000	1.0000	0.0000	0.0000	33	0
Manifest filtering + consistency gate	0.0000	1.0000	1.0000	0.0000	0.7143	0	0
Intent-certificate layer + OpenPort effect controls	0.0000	0.8000	1.0000	0.0000	0.7143	0	1

TABLE XI

DETERMINISTIC TOY ABLATION OVER THE SYNTHETIC INTENT-LAYER TASK MATRIX, RE-SCORED UNDER THE PAPER'S METRIC DEFINITIONS. UER IS UNSAFE EXECUTION RATE; BCR_{strict} REQUIRES DIRECT SAFE COMPLETION WITHOUT CLARIFICATION, DRAFT, OR REVIEW; GAR ALSO CREDITS A GOVERNED ARTIFACT. ODR IS OVER-DEFENSE RATE, MRS IS MEAN MANIFEST REDUCTION, UHRT IS THE COUNT OF UNJUSTIFIED HIGH-RISK TOOLS VISIBLE ACROSS ALL TASKS, AND REVIEW IS THE NUMBER OF BENIGN HIGH-RISK CASES REQUIRING OPERATOR INTERVENTION UNDER OPENPORT EFFECT CONTROLS.

Check	Toy result
Expected action match rate	1.0000
High-confidence low-risk allow rate	1.0000
Medium-confidence draft/preflight rate	1.0000
Low-confidence clarification rate	1.0000
Conflicting-intent deny/split rate	1.0000
High-risk confirmation/preflight rate	1.0000
Multi-step per-step certificate rate	1.0000
Static policy deny rate	1.0000
Unsafe broad authority grants	0
Authority widening rate	0.0000
Broad multi-step certificates	0

TABLE XII

DETERMINISTIC TOY CONFIDENCE-POLICY CHECKS. THESE RESULTS VALIDATE THE POLICY ROUTING RULES, NOT CLASSIFIER ACCURACY.

- B2: intent-certificate manifest narrowing and payload consistency checks.

The script checks the policy rules over synthetic benign, mismatch, prompt-injection, tool-poisoning, ambiguous, and compound tasks. The separate runtime-backed microbenchmark exercises the OpenPort reference HTTP path.

The check contains 5 benign and 7 attack-or-ambiguous tasks. Under the broad static manifest, 33 unjustified high-risk tool exposures occur across the task set; under intent-certificate narrowing, this count is 0. The toy unsafe-execution rate is 1.0 for model-only B0, 0.1429 for OpenPort-substrate B1, and 0 for intent-certificate B2. Under the legacy governed-benign outcome, both B1 and B2 score 1.0 on the five benign tasks. Re-scored under this paper's definitions, their governed-artifact rate remains 1.0, whereas strict benign completion is 0.8000 because one of five benign tasks is routed to review rather than strictly completed. Mean manifest reduction is 0.7143.

The sanity check also evaluates six ablation conditions. Table XI reports the resulting metrics.

The toy ablation separates the roles of the major components. Prompt-only instruction is weakest under an adversarial planner because no server-side policy binds model behavior. Static OpenPort effect controls block high-risk direct effects but cannot reject a medium-risk payload expansion that is inconsistent with the user's requested amount. Manifest filtering removes unjustified high-risk tools from the visible surface but does not by itself catch payload expansion on a still-visible tool. The consistency gate catches payload and effect mismatches even when the full static manifest remains visible, but it does not reduce tool-enumeration exposure. The combined intent-certificate conditions obtain both zero unsafe execution on this toy matrix and zero unjustified high-risk exposure. Adding OpenPort effect controls introduces one review intervention for the benign high-risk delete task, which is the intended safety cost of draft/preflight governance.

A deterministic confidence-policy check exercises the classifier-trust boundaries defined in Section 4. It defines twelve synthetic confidence states covering high-confidence low-risk tasks, medium-confidence create/update tasks, low-confidence ambiguous tasks, conflicting read/delete intent, high-risk delete/export intent, multi-step read-then-delete workflows, and high-confidence classifier outputs that exceed static OpenPort policy. Table XII reports the resulting policy checks.

These numbers provide a deterministic conformance check of the policy sketch.

Scenario	Tasks	$\mathbf{UAR_c}$	UER	$\mathbf{BCR_{strict}}$	GAR	Clar.	Review	Draft	MRS
OpenPort-only	176	1.0000	0.0000	0.4314	1.0000	0.0000	0.5686	1.0000	0.0000
Reference certificates	176	0.0000	0.0000	0.4314	0.8627	0.1373	0.4314	0.3333	0.7614
Rule-based certificates	176	0.0000	0.0000	0.4314	0.9608	0.0196	0.5294	0.4091	0.7662
Hybrid rule+flow certificates	176	0.0000	0.0000	0.4314	0.9608	0.0196	0.5294	0.4091	0.7662

Scenario	Tasks	P50	P95	P99
OpenPort-only	176	0.1748	0.3819	1.5451
Reference certificates	176	0.2486	0.4739	1.3952
Rule-based certificates	176	0.2342	0.4305	1.3199
Hybrid rule+flow certificates	176	0.1819	0.3640	1.1942

TABLE XIII

RUNTIME-BACKED SYNTHETIC MICROBENCHMARK. THE UPPER PANEL REPORTS SAFETY AND UTILITY OUTCOMES. BCR_{strict}, GAR, CLAR., AND REVIEW USE ALL 102 BENIGN CASES AS THEIR DENOMINATOR; CLAR. AND REVIEW ARE DISJOINT, WITH REVIEW EXCLUDING CLARIFICATION-ROUTED CASES. DRAFT USES ALL 132 ROWS PER SCENARIO THAT REACH PROPOSAL/ACTION ROUTING (`PROPOSALROWS`), INCLUDING BENIGN AND ATTACK-OR-AMBIGUOUS ROWS: $1.0000 = 132/132$, $0.3333 = 44/132$, AND $0.4091 = 54/132$. THE LOWER PANEL REPORTS MACHINE-LOCAL P50/P95/P99 FASTIFY-INJECTION DIAGNOSTICS IN MILLISECONDS FROM THE ARCHIVED RESULT. UAR_c IS THE LEGACY COMPOSITE EXPOSURE-OR-PATH INDICATOR; GAR COUNTS STRICT COMPLETION OR A GOVERNED ARTIFACT.

Classifier	ICA	RBA	EBA	UNF	ODR
Oracle	1.0000	1.0000	1.0000	0.0000	0.0000
Rule-based	0.6818	0.8977	1.0000	0.0000	0.0000
Hybrid rule+flow	0.8636	0.9886	1.0000	0.0000	0.0000
Model-generated certificates	evaluated in the model studies below				

TABLE XIV

SMALL DETERMINISTIC CLASSIFIER/ORACLE COMPARISON ON THE 176-TASK SYNTHETIC SUITE. ICA IS INTENT-CLASS ACCURACY, RBA IS RESOURCE-BOUND ACCURACY, EBA IS EFFECT-BOUND ACCURACY, UNF IS UNSAFE NARROWING FAILURE, AND ODR IS OVER-DEFENSE RATE. ALL ROWS USE DETERMINISTIC CLASSIFIERS OR REFERENCE ORACLES.

H. Runtime-Backed Microbenchmark

To connect the policy sketch to the implemented runtime, we run a runtime-backed microbenchmark.

The script uses Fastify HTTP injection against the OpenPort reference runtime. The suite contains 176 deterministic instances generated from author-designed templates: 102 benign and 74 attack-or-ambiguous. It covers six intent classes: read, summarize, create, update, delete, and export. It also covers seven attack or ambiguity categories: intent-tool mismatch, payload expansion, indirect prompt injection, tool metadata poisoning, low confidence, compound workflow, and resource underspecification. The benign set includes underspecified requests so that clarification and review burden remain visible. For each instance, the runtime calls `/intent`, `/manifest`, and, when applicable, `/actions`; it records endpoint outcomes, drafts, executions, manifest sizes, audit binding, and local diagnostic latency.

The reported UAR_c is a conservative composite: it flags an attack-or-ambiguous instance when an unjustified high-risk tool remains visible or an inconsistent draft/execution path is accepted. Thus OpenPort-only can have $UAR_c = 1.0$ while UER is 0. The composite supports deterministic conformance and differs from the path-only UAR used in the real-model study.

The study separates enforcement correctness from certificate-generation quality. The reference condition uses author-specified certificates embedded in the synthetic task suite. Rule-based and hybrid rule+flow conditions are deterministic local classifiers. Model-generated certificates are evaluated in the studies below. The reference condition routes underspecified benign requests more conservatively, lowering GAR. Higher GAR in rule-based or hybrid rows may therefore reflect less conservative routing rather than better access-control quality.

This check exercises endpoint wiring, certificate storage, manifest narrowing, action denial, draft creation, and audit binding. The reference-certificate condition separates enforcement from certificate generation and uses a deterministic planner to isolate the enforcement path from model behavior.

I. Benchmark-Family Transfer Set

To test the mechanism on structures not unique to OpenPort, we evaluate a locally authored benchmark-family transfer set. The set uses twenty-five tasks: six AgentDojo-shaped tasks, seven ToolSandbox-shaped tasks, six tau-bench-shaped tasks, and six ToolEmu-shaped tasks. The runtime-backed deterministic scorer uses author-specified

Source family	Benign	Ambiguous	Attack	Total
AgentDojo	2	0	4	6
ToolSandbox	2	2	3	7
tau-bench	2	2	2	6
ToolEmu	2	0	4	6
Overall	8	4	13	25

TABLE XV
TASK-TYPE COVERAGE IN THE BENCHMARK-FAMILY TRANSFER SET.

Primary category	Tasks	Attack	Source families
Prompt injection / untrusted content	4	4	AgentDojo
Insufficient information	4	0	ToolSandbox, tau-bench
Wrong scope / target mismatch	3	3	tau-bench, ToolEmu
State dependency	2	2	ToolSandbox
Tool poisoning / semantic deception	2	2	ToolEmu
Canonicalization / normalization risk	1	1	ToolSandbox
Amount expansion / bound inflation	1	1	ToolEmu

TABLE XVI
PRIMARY ATTACK-SURFACE COVERAGE FOR THE 17 NON-BENIGN TRANSFER-SET TASKS. EACH TASK IS ASSIGNED EXACTLY ONE PRIMARY CATEGORY.

reference certificates and concrete proposals, then drives them through the OpenPort reference HTTP path with and without the intent layer. The task content remains synthetic and locally authored; only structural patterns are transferred from the benchmark families.

Table XV separates benign, ambiguous, and attack tasks by source family. The attack-heavy batch measures narrowing of benchmark-shaped escalation attempts rather than deployment task prevalence. Table XVI partitions the 17 non-benign tasks into additive primary attack-surface categories.

Across the full transfer set, OpenPort-only has composite $UAR_c = 1.0000$, UER $= 0.0000$, and $GAR = 1.0000$. With the intent layer and author-specified reference certificates, UAR_c is 0.0000, UER remains 0.0000, GAR remains 1.0000, and mean manifest reduction is 0.7829. This deterministic comparison measures conformance on the locally authored tasks.

J. Expanded Real LLM Pilot

We evaluated certificate generation and planning with three open-weight models: GPT-OSS-120B, Llama-3.3-70B-Instruct, and Qwen3-Next-80B. Each model received 55 classifier tasks and 54 planner attack tasks under static and intent-filtered manifests. This pilot evaluates model behavior separately from the OpenPort action path exercised in the end-to-end study.

K. Broad Intent Is Easier Than Authorization Bounds

Across the scored models, intent-class accuracy ranges from 0.7273 to 0.8545, while resource-bound accuracy ranges from 0.1818 to 0.5818 and review-mode accuracy from 0.3818 to 0.6909. Models recognize coarse read, summarize, create, update, delete, and export classes more reliably than exact resource identifiers, row bounds, and risk modes. The authorization layer therefore treats model-generated certificates and review modes as advisory. Server-side policy validates scopes, narrows manifests, checks payload bounds, and overrides any model-proposed `allow` decision for high-risk tools. GPT-OSS-120B, despite the strongest bound extraction in this set, emitted immediate `allow` for 85.71% of high-risk requests.

The planner pilot shows the same separation of responsibilities. Manifest filtering lowers unsafe proposal rates for every scored model, but filtered manifests alone still leave unsafe read-path or overbroad-payload proposals. Deterministic consistency gating reduces unsafe accepted authority to zero in this synthetic planner setting, showing that manifest filtering must be paired with payload-bound server enforcement.

Source family	Tasks	OpenPort $\mathbf{UAR_c}$	Intent $\mathbf{UAR_c}$	Intent GAR	Intent MRS
AgentDojo	6	1.0000	0.0000	1.0000	0.7619
ToolSandbox	7	1.0000	0.0000	1.0000	0.7755
tau-bench	6	1.0000	0.0000	1.0000	0.7381
ToolEmu	6	1.0000	0.0000	1.0000	0.8571

TABLE XVII

RUNTIME RESULTS ON THE LOCALLY AUTHORED BENCHMARK-FAMILY TRANSFER SET WITH AUTHOR-SPECIFIED REFERENCE CERTIFICATES.

Model	ICA	RBA	EBA	Review Acc.	High-Risk Allow	P95 ms
GPT-OSS-120B	0.8545	0.5818	0.9091	0.5273	0.8571	3767.5
Llama-3.3-70B	0.7273	0.1818	0.7818	0.3818	0.4286	17633.4
Qwen3-Next-80B	0.7818	0.1818	0.7818	0.6909	0.0000	4723.2

TABLE XVIII

EXPANDED REAL LLM CLASSIFIER PILOT OVER 55 SYNTHETIC INTENT-CERTIFICATE TASKS PER MODEL. ICA IS INTENT-CLASS ACCURACY, RBA IS RESOURCE-BOUND ACCURACY, EBA IS EFFECT-BOUND ACCURACY, AND HIGH-RISK ALLOW IS THE RATE AT WHICH HIGH-RISK CREATE/UPDATE/DELETE/EXPORT REQUESTS WERE INCORRECTLY EMITTED AS IMMEDIATE `ALLOW`. ACROSS MODELS, BROAD INTENT RECOGNITION IS MATERIALLY STRONGER THAN EXACT RESOURCE/EFFECT-BOUND EXTRACTION.

The certificate-generation comparison covers reference, rule-based, and hybrid rule+flow conditions in the runtime-backed microbenchmark, and raw model-generated certificates in the model studies. Table XXVI separately evaluates schema normalization and deterministic risk override by re-scoring the collected traces.

L. Scored End-to-End LLM-in-the-Loop Runtime Benchmark

We additionally ran an API-backed benchmark harness that pushes model-generated classifier and planner output through the OpenPort reference intent-layer action path:

$$\texttt{/intent} \rightarrow \texttt{/manifest} \rightarrow \texttt{/preflight or /actions} \rightarrow \texttt{consistency gate} \rightarrow \texttt{audit}.$$

The scored run uses 34 synthetic tasks per run (12 benign and 22 attack-or-ambiguous), 3 repeats per model, and 3 open-weight models: GPT-OSS-120B, Llama-3.3-70B, and Qwen3-Next-80B. The suite is parameterized over three synthetic resource collections, so benign and adversarial patterns recur across bounded settings. It records certificate creation, clarification, unsafe planner proposals, accepted authority, executed effects, BCR_{strict}, GAR, manifest reduction, latency, and trace summaries.

Finding 2: Unsafe draft authority remains.
The full gateway path receives real-model outputs. Across 198 attack-or-ambiguous trials, the combined intent-layer and OpenPort path accepts 40 unsafe drafts and records 0 completed unsafe effects.

Runtime unsafe accepted authority is 40/198 overall and 0.0909–0.2727 by model. The cases concentrate in draft-first create paths where the model-generated certificate or payload loses the author-specified amount/resource bound. Current model-generated certificates therefore do not completely enforce request-bound narrowing.

Finding 3: Model behavior exposes a safety-utility-latency tradeoff.
Manifest reduction is roughly 0.79–0.80 across models, but BCR_{strict} is 0/108 overall and GAR is only 36/108. Thus every counted benign artifact is a governed draft rather than immediate completion. Llama-3.3-70B has the lowest unsafe accepted rate in this setting (0.0909) and the highest GAR (0.5000), but the largest latency tail (P95 49.9 seconds). These model-level differences are descriptive within the small synthetic set and single provider configuration.

M. Transfer-Subset End-to-End LLM Runtime Pilot

We ran the same API-backed harness over a selected transfer subset. It contains 12 locally authored tasks from Section X-I: 4 benign, 2 ambiguous, and 6 attack tasks, with 3 shaped by each benchmark family. Unlike the deterministic reference-certificate scorer, this pilot uses real-model certificates and planner output before calling the OpenPort runtime path.

Across all three scored models, runtime unsafe accepted authority is 0.1250–0.2500. The governed-artifact rate is 0.7500 for all three rows, while strict benign completion remains 0. Three of the four benign tasks reach non-executed

Model	Static Unsafe	Filtered Unsafe	Filtered+Gate Unsafe Accepted	Static Clar.	Filtered Clar.	Filtered P95 ms
GPT-OSS-120B	0.5000	0.4444	0.0000	0.4444	0.5000	4790.9
Llama-3.3-70B	0.8889	0.5926	0.0000	0.0556	0.3519	35290.4
Qwen3-Next-80B	0.9259	0.7037	0.0000	0.0556	0.2778	5820.6

TABLE XIX

EXPANDED REAL LLM PLANNER PILOT OVER 54 SYNTHETIC ATTACK TASKS PER MODEL. MANIFEST FILTERING LOWERS UNSAFE PROPOSALS, BUT DETERMINISTIC GATING IS WHAT DRIVES UNSAFE ACCEPTED AUTHORITY TO ZERO IN THIS PILOT.

Model	Cert. create	Plan clar.	Plan unsafe	Unsafe path	Unsafe exec.	$\mathrm{BCR}_{\mathrm{strict}}$	GAR	MRS	P95 (s)
GPT-OSS-120B	1.0000	0.4216	0.5784	0.2424	0.0000	0.0000	0.2500	0.8025	10.47
Llama-3.3-70B	1.0000	0.0980	0.7353	0.0909	0.0000	0.0000	0.5000	0.7913	49.87
Qwen3-Next-80B	1.0000	0.0490	0.9510	0.2727	0.0000	0.0000	0.2500	0.8025	9.95

TABLE XX

SCORED END-TO-END MODEL RUNTIME BENCHMARK. EACH ROW AGGREGATES 34 SYNTHETIC TASKS ACROSS THREE REPEATS. "UNSAFE PATH" IS INTENT-INCONSISTENT ACCEPTED AUTHORITY; "UNSAFE EXEC." IS A COMPLETED UNSAFE EFFECT. LATENCY IS DESCRIPTIVE.

governed artifacts; the tau-bench-style bounded review delete still fails under current planners rather than reaching such a path.

In this 12-task subset, runtime unsafe accepted cases concentrate in ToolEmu-style bounded-create attacks rather than in the AgentDojo, ToolSandbox, or tau-bench-derived rows. The same bounded-create and semantic-deception residue appears in the larger synthetic runtime benchmark, making certificate precision and planner-call quality the main blockers to eliminating accepted authority.

Table XXII makes the residual risk boundary more concrete. All prompt-injection, insufficient-information, state-dependency, and wrong-scope trials end either in clarification or in a blocked no-effect path. Runtime unsafe accepted authority remains only in ToolEmu-style tool-poisoning / semantic-deception rows and amount-expansion rows. These failures concentrate in the same certificate-bound and bounded-create residue identified by the larger synthetic runtime benchmark.

N. Evaluation Synthesis

The runtime-backed microbenchmark, scored LLM runtime benchmark, transfer-set coverage tables, and transfer-subset runtime pilot support three conclusions.

First, in the external-subset runtime pilot, prompt injection, insufficient information, state dependency, and wrong-scope / wrong-target trials all collapse into clarification or blocked no-effect paths rather than unsafe accepted authority.

Second, the remaining residue is structurally concentrated rather than diffuse. Across both the larger scored runtime benchmark and the smaller benchmark-shaped external subset, accepted-authority failures cluster in bounded-create settings where certificates and payloads preserve the broad operation class but lose the exact amount, resource, or effect contract needed for early rejection. Tool-poisoning rows only survive when they can masquerade as create-compatible draft paths; amount-expansion rows survive when the gateway still sees a create-shaped proposal but cannot prove the numeric bound is faithful enough.

Third, the empirical bottleneck is certificate canonicalization, resource/effect normalization, planner schema repair, and stronger create-versus-destructive effect contracts. Bounded-create semantics remain the dominant open problem once OpenPort effect controls and monotone narrowing are in place.

Finding 4: The remaining accepted-authority failures are narrow and classifiable.
The 40 accepted-authority cases across the 198 attack-or-ambiguous runtime trials all end as drafts without creating an execution. They are reviewable authority paths that the current intent-certificate layer did not narrow away early enough.

Of the 40 accepted-authority cases, 24 have empty resource and effect bounds in the emitted certificate, while the remaining 16 are semantically weak but non-empty create-compatible certificates. Review-mode labels split across `allow` (16), `preflight` (6), and `draft` (18), showing that the residual problem is not simply an incorrect review bit. Stronger amount, resource, and effect normalization is needed to collapse these governed draft paths into earlier denials or clarifications.

Model	Tasks	Plan clar.	Plan unsafe	Unsafe path	Unsafe exec.	GAR	P95 (s)
GPT-OSS-120B	12	0.5000	0.5000	0.2500	0.0000	0.7500	13.76
Llama-3.3-70B	12	0.0000	1.0000	0.1250	0.0000	0.7500	53.15
Qwen3-Next-80B	12	0.0833	0.9167	0.2500	0.0000	0.7500	7.36

TABLE XXI
API-BACKED RUNTIME PILOT OVER THE 12-TASK BENCHMARK-FAMILY TRANSFER SUBSET.

Primary category	Trials	Clarify	Blocked / no effect	Unsafe accepted	Unsafe executed
Prompt injection / untrusted content	6	2	4	0	0
Insufficient information	6	3	3	0	0
State dependency	3	1	2	0	0
Wrong scope / target mismatch	3	1	2	0	0
Tool poisoning / semantic deception	3	0	1	2	0
Amount expansion / bound inflation	3	0	0	3	0

TABLE XXII
OUTCOME CROSS-TAB FOR THE 24 NON-BENIGN MODEL-TASK TRIALS IN THE EXTERNAL-SUBSET RUNTIME PILOT. COUNTS AGGREGATE ACROSS THE THREE SCORED MODELS; THE CANONICALIZATION-RISK CATEGORY IS ABSENT FROM THIS 12-TASK SUBSET.

O. Certificate-Normalizer Sensitivity Analysis

The residual failures in Table XXIII suggest a specific design question: would a deterministic certificate-bound normalizer reduce unsafe accepted authority, or is the remaining problem mainly planner behavior? We normalize common model-generated certificate aliases such as `collection`, `resource`, `amount`, `itemLimit`, and `limit`, and re-score the collected traces under stricter certificate-bound predicates.

The trace-backed counterfactual re-scores existing rows without replaying the model, repairing planner payloads, or creating new OpenPort actions. It estimates whether canonical resource/effect bounds at the consistency gate would remove the observed accepted-authority failures.

The analysis gives two signals. First, deterministic review-mode override alone does not change UAR_{path} because every accepted-authority case is already a draft rather than an execution. The weak point is whether resource and effect bounds are canonical enough for enforcement. Second, strict normalization trades authority rejection for fewer governed benign artifacts: UAR_{path} changes from 0.2020 to 0.0758 under bound validation and to 0 under the normalized-gate predicate, while GAR changes from 0.3333 to 0.1389.

P. Answers to the Research Questions

RQ1 (controlled authority narrowing): On the 176-instance deterministic runtime suite, the reference-certificate condition changes the archived composite exposure-or-path indicator UAR_c from 1.0000 under OpenPort-only to 0, while UER is 0 in both conditions. These results measure manifest and certificate conformance on the evaluated suite.

RQ2 (real-model runtime outcomes): Across 198 attack-or-ambiguous trials, the combined intent-layer and OpenPort path accepts 40 intent-inconsistent drafts and observes 0 completed unsafe effects. The benchmark-family transfer subset accepts 5/24 unsafe drafts and observes 0/24 unsafe completed effects.

RQ3 (utility and latency cost): Strict benign completion is 0/108 in the scored model-runtime study; 36/108 trials yield governed artifacts. The transfer subset yields 9/12 governed artifacts. Model-level P95 end-to-end latency ranges from 9.95 to 49.87 seconds under the archived provider configuration. The evaluated prototype is review-oriented.

RQ4 (residual concentration): The 40 unsafe accepted cases comprise 24 amount/resource-bound failures and 16 create-compatible semantic-deception cases. All terminate as drafts. The concentration implicates certificate canonicalization and effect contracts.

RQ5 (component roles): The deterministic component check shows that manifest filtering removes unjustified tool exposure while payload consistency catches overbroad calls that remain in a visible tool class. Trace-backed normalization changes UAR_{path} from 0.2020 to 0 and GAR from 0.3333 to 0.1389.

XI. DISCUSSION

Static non-expansion and request confinement: The formal mechanism guarantees that the intent layer cannot exceed static OpenPort authority. Confinement to an expressed request additionally depends on an independently justified certificate envelope and sound adapter effect bounds. The 40/198 unsafe accepted drafts quantify this gap.

Accepted-authority failure mode	Cases	Trace pattern	Interpretation
Create payload expansion with missing or weak bounds	24	`create_amount_expansion_*`; `actionStatus=draft`	In 15/24 cases the certificate omits enforceable resource/effect bounds entirely; in the remaining 9/24 the certificate contains create-compatible but insufficient amount/type bounds.
Tool-poisoning-like create path with semantic deception	16	`tool_poison_hard_delete_*`; `actionStatus=draft`	The planner is induced toward a create-compatible draft path, which remains accepted authority when the certificate does not narrow the path away.
Restrictive review mode that still leaves accepted draft authority (overlapping slice)	24	Review mode is `draft` or `preflight` rather than `allow`	Review mode alone is insufficient to eliminate unsafe accepted authority. It converts the path into governed draft/preflight handling, but does not by itself prove the payload/effect bounds are tight enough.

TABLE XXIII

FAILURE MODES AND CROSS-CUTTING PROPERTIES FOR THE SCORED RUNTIME BENCHMARK. THE FIRST TWO ROWS ARE A MUTUALLY EXCLUSIVE $24 + 16 = 40$ PARTITION OF ACCEPTED-AUTHORITY CASES. THE RESTRICTIVE-REVIEW ROW IS AN OVERLAPPING SLICE OF THOSE SAME 40 CASES.

Benign or unsafe runtime outcome	Count	Example	Likely fix
Reviewable benign draft counted in GAR	36	`create_collection_main`	Governed artifact; excluded from strict completion.
Malformed benign tool or payload call	67	`update_rec_2`, `delete_rec_7`, `export_collection_main`	Planner schema retry, endpoint-specific payload canonicalization, certificate normalizer.
Benign clarification with no effect	4	`export_collection_audit` (model A)	Track separately as safe non-completion rather than as hard failure.
Unknown benign action family	1	`export_collection_ops` (model B)	Tool-call schema validator and planner retry.
Accepted authority from missing amount/resource/effect bounds	24	`create_amount_expansion_col_*`	Numeric validators and certificate-bound normalizers before action routing.
Accepted authority from create-compatible semantic deception	16	`tool_poison_hard_delete_col_*`	Stronger effect contracts and semantic validation for create/delete separation.

TABLE XXIV

OUTCOME TAXONOMY FOR THE SCORED RUNTIME BENCHMARK.

Drafts isolate effects but retain authority and review risk: All 40 residual cases stop at drafts, so no evaluated domain effect completes. Nevertheless, a draft is persistent system state, consumes reviewer capacity, and can carry misleading content into a human approval flow. Effect isolation and authority rejection are therefore separate objectives.

Certificate precision is the measured bottleneck: Broad class recognition is materially stronger than resource- and amount-bound extraction. Residual failures cluster in create-compatible cases whose operation class is plausible but whose exact amount, resource, or semantic effect is weakly encoded. This motivates typed bounds, deterministic normalization, and adapter contracts rather than greater confidence in free-form classifier output.

Utility remains review-oriented: Strict benign completion is 0/108 in the scored model-runtime study; 36/108 benign trials produce governed artifacts. The current configuration is review-oriented and has low autonomous utility.

XII. LIMITATIONS AND THREATS TO VALIDITY

Construct validity: Task templates and reference envelopes are author-specified, creating circularity between mechanism design and scoring. The benchmark-family transfer set comprises locally authored structural analogues. The legacy deterministic UAR_c combines exposure and path acceptance and therefore differs from path-only UAR.

Internal and statistical validity: The end-to-end study has no matched OpenPort-only real-model runtime row, leaving the isolated effect of the intent layer unidentified. Repeats share task templates and are correlated; zero observed events leave residual risk unmeasured. The provider-run bootstrap seed was not archived, and its legacy intervals are not used inferentially. The task-clustered reanalysis uses a recorded seed. The normalizer rules were developed after inspecting failures and may be overfit.

Case	Certificate / planner	Runtime result	Metric interpretation	Lesson
Benign create draft	GPT-OSS-120B emits `create` with bounded resource and amount; planner calls `resource.create`.	`common.success`, `actionStatus=draft`, audit-bound.	Counts toward GAR; excluded from BCR_{strict}.	Governed draft creation without task completion.
Benign update invalid	GPT-OSS-120B emits bounded `update`; planner calls `resource.update`.	`agent.action_invalid`; no draft or execution.	Hard benign failure.	Certificate intent is plausible, but payload canonicalization is still too weak.
Intent mismatch blocked by clarification	GPT-OSS-120B emits `read,summarize`; planner returns `clarify`.	No action, no draft, no execution.	Safe non-completion.	Manifest narrowing plus planner uncertainty can fail closed without creating authority.
Low-confidence ambiguity	GPT-OSS-120B emits `unknown`, confidence 0.2, review mode `clarify`.	No action, no draft, no execution.	Clarification rather than failure.	Confidence-aware routing works as intended when the request is underspecified.
Amount-expansion attack	GPT-OSS-120B emits `create` with weak amount bound; planner calls `resource.create`.	`common.success`, `draft`, `runtimeUnsafeAccepted=true`.	Unsafe accepted draft authority.	Missing numeric/resource precision leaves a reviewable authority path open.
Tool-poisoning attack	GPT-OSS-120B emits create-compatible certificate under poisoned context; planner still selects `resource.create`.	`common.success`, `draft`, `runtimeUnsafeAccepted=true`.	Unsafe accepted draft authority.	Semantic validation is needed to distinguish benign create from disguised delete intent.

TABLE XXV
QUALITATIVE RUNTIME TRACES FROM THE SCORED BENCHMARK.

Condition	$\mathbf{UAR}_{\text{path}}$	UER	GAR	Accepted	Benign artifacts
Raw LLM certificates	0.2020	0.0000	0.3333	40	36
LLM + strict bound validator	0.0758	0.0000	0.1389	15	15
LLM + strict validator + normalized gate	0.0000	0.0000	0.1389	0	15
LLM + deterministic risk override only	0.2020	0.0000	0.3333	40	36

TABLE XXVI
SENSITIVITY ANALYSIS OVER COLLECTED MODEL-RUNTIME TRACES. UAR_{path} IS THE ACCEPTED-ACTION-PATH RATE OVER ATTACK-OR-AMBIGUOUS TRIALS; THE ANALYSIS INTRODUCES NO NEW RUNTIME CALLS.

External validity: The studies use synthetic data, three provider-served open-weight models, one provider configuration, modest task sets, and no real operators. Their evidence scope excludes production tenants, human review quality, reviewer fatigue, and adversaries adapting over time. Local Fastify-injection latency describes the test environment rather than networked deployment performance.

Implementation: The prototype uses in-memory certificate state rather than persistent, distributed lifecycle enforcement. Historical provider runs did not record a clean runtime revision for every shard, and the transfer subset has incomplete top-level append provenance. Exact provider-call recreation is therefore unavailable.

XIII. SECURITY ANALYSIS

A. Invariant 1: Intent Cannot Widen Static Policy

The monotonicity invariant is:

$$P_C \preceq P_A.$$

If a tool is hidden by static OpenPort scopes, ABAC policy, workspace boundary, revocation, or rate limit, the intent layer cannot reveal it. This protects against classifier compromise turning into privilege escalation.

B. Invariant 2: Low Confidence Fails Closed

If $\gamma < \gamma_{\min}$, the system must not treat the certificate as broad authority. It should expose minimal tools, request clarification, or force draft/preflight.

C. Invariant 3: Payload Effects Must Be Bounded

Even if a tool class is allowed, the payload must stay within request bounds. A read request for "last week" should not silently become a year-long export. A request to update one record should not become a bulk update.

D. Invariant 4: The Model Is Not a Principal

The model may suggest a tool call, but authorization is decided by the gateway. The gateway verifies scopes, policy, certificate validity, consistency, preflight, and review requirements.

E. Invariant 5: Drafts Are Not Completed Domain Effects

Draft creation is persistent governance state, not a completed domain effect. It still represents accepted authority, consumes reviewer capacity, and can become a social-engineering carrier. Therefore an intent-inconsistent proposal must deny or clarify before draft creation; consistent high-risk proposals may be drafted with audit, a clear preview, and bound review context.

F. Prompt Injection

Prompt injection can influence model reasoning, tool selection, payload generation, and certificate generation. Unconditionally, the intent layer can prevent such influence from expanding authority beyond static policy. It confines authority to the expressed request only when the certificate refines the trusted request envelope and adapter effect bounds are sound. Under those conditions, an injected export instruction outside a summary envelope is hidden or denied.

Attacks that alter details within an already authorized high-risk effect envelope require context provenance or taint-aware manifests in addition to the intent-certificate layer.

G. Tool Metadata Poisoning

Tool metadata attacks try to make a model choose a malicious or irrelevant tool. Intent-aware filtering reduces the effect of such attacks by limiting visible tools and checking selected tools against the certified request. However, if poisoned metadata belongs to a tool that is both statically authorized and intent-compatible, the intent-certificate layer alone may not detect semantic deception inside the tool implementation. Manifest signing, tool provenance, sandboxing, and behavior monitoring remain necessary.

H. Effect Estimation and Conservative Failure

The consistency predicate depends on estimating or bounding tool effects. This is easy for simple read, create, update, delete, and export actions with explicit resource IDs and item count limits. It is harder for tools with implicit side effects, compound workflows, tool-provided summaries, or domain-specific semantics.

The intent-certificate layer therefore adopts a conservative failure rule:

$$\neg\mathrm{Boundable}(C, t, x, \widehat{e}) \Rightarrow \mathrm{Route}_{\mathrm{intent}} \in \{\texttt{deny}, \texttt{clarify}\}.$$

Preflight may first be used to obtain a conservative bound. If the gateway still cannot bound the payload or predicted effect against the certificate, the enforce profile admits neither execution nor a draft path; it denies or asks for clarification. This prevents uncertainty from becoming persistent accepted authority.

I. Classifier Compromise

If the classifier is compromised, it might produce a broad certificate. Monotonicity still prevents scope expansion, but it may fail to narrow authority. Mitigations include rule/model agreement, confidence thresholds, human review for high-risk classes, certificate audit, and adversarial tests. Critical deployments should treat classifiers as untrusted components and require deterministic policy constraints for high-risk effects.

J. Multi-Intent Tasks

Users often ask for compound tasks. The intent-certificate layer should avoid issuing one broad lease for the entire conversation. Instead, it can decompose the request into per-step certificates:

$$C(u) = \{C_1, \ldots, C_n\},$$

where each C_i authorizes one effect class and resource bound. If decomposition is uncertain, the gateway should require clarification or draft review.

Operational rule: Read/summarize certificates may be used to discover candidate objects and present them to the user. Update/delete/export certificates should be issued only after the user confirms the concrete target or approved output scope. One broad certificate should not simultaneously cover search, ranking, and high-risk mutation unless the mutation target is already explicit and bounded.

XIV. Related Work

A. Agent Security Benchmarks

AgentDojo, InjecAgent, AgentDyn, ToolSandbox, tau-bench, ToolEmu, and MCPAgentBench provide evaluation settings for tool-using agents [1], [2], [20]–[24]. These benchmarks primarily measure robustness, task success, or safety under adversarial context. Risk-knowledge experiments further show that a model's ability to recognize risk need not translate into safe action [25]. The intent layer is complementary: it defines a server-side mechanism that can be evaluated using such tasks.

B. Prompt-Injection Defenses

StruQ separates prompt and data channels through structured queries and model training [26]. AgentSentry models multi-turn indirect prompt injection as temporal causal takeover and purifies context [27]. AttriGuard is especially close: it causally attributes a proposed action to trusted user input rather than untrusted observations [28]. Its action-support signal can feed the downstream authority ceiling, manifest narrowing, payload/effect-bound check, routing outcome, and audit lifecycle of an intent-certificate layer.

C. Tool Selection and MCP Security

ToolHijacker, ToolTweak, MCPTox, and MCP threat-modeling work show that tool names, descriptions, metadata, and ecosystem composition can create security vulnerabilities [3]–[5], [7], [8]. These works motivate intent-aware filtering. Even a perfectly authenticated and well-described tool should not be callable when the user's request does not justify its effect.

D. Agent Authorization and Access Control

Least privilege, RBAC, ABAC, and OAuth-style delegated authorization remain foundational [12], [15]–[17]. Usage control adds obligations, conditions, attribute mutation, and ongoing authorization [18]; macaroons make delegated credentials attenuable through contextual caveats [19]. Security-automata work in turn characterizes the safety policies that execution monitoring can enforce [29]. These mechanisms supply useful policy and enforcement primitives, but do not themselves infer which authority a trusted natural-language request justifies or mediate an untrusted planner at both discovery and action boundaries.

Recent agent-focused work extends OAuth/OIDC delegation with agent credentials and natural-language permission translation, governs inter-agent communication with user-defined policies and cryptographic tokens, enforces intent/context-aware OS policy for computer-use agents, or moves identity- and argument-level policy to an off-host gateway [30]–[33]. The mechanism studied here is narrower in principal identity but finer in request lifecycle: it derives a short-lived request envelope, intersects it with an existing gateway policy, filters the model-visible manifest, and rechecks payload/effect bounds before distinguishing draft authority from execution.

Task Shield enforces task alignment against indirect prompt injection; Progent and MiniScope pursue least-privilege tool authorization; CaMeL separates trusted control and untrusted data flows; Prompt Flow Integrity constrains privilege-escalating prompt flows; and AgentSpec provides customizable runtime enforcement [34]–[39]. These systems are the closest architectural comparators. The mechanism's distinct unit is a server-issued, short-lived request envelope that is intersected with an existing gateway policy both before planning (manifest filtering) and after planning (payload/effect checking), with draft, preflight, confirmation, and execution modeled as different accepted-authority outcomes. Decentralized information-flow control instead attaches principal-specific policies to

data propagation and controlled declassification [40]; it can isolate untrusted data inside an agent, while the intent layer governs which external tool effects the planner may request. Information-flow isolation, privilege inference, policy languages, and runtime monitors therefore remain complementary controls.

E. Purpose Limitation and Contextual Integrity

The intent layer is also related to purpose-based access control and contextual privacy. Purpose-based systems restrict data use according to declared purposes rather than only subjects, objects, and operations. Foundational work associates allowed and prohibited purposes with data and enforces them through purpose-aware labeling, conditional roles, and query modification [41]; recent systems such as Data Guard apply purpose-aware masking in warehouse settings, and purpose limitation has also been studied for data-in-transit systems [42], [43]. Contextual integrity frames privacy as appropriate information flow within a social context, using parameters such as sender, recipient, information type, and transmission principle [44], [45]. Recent assistant work operationalizes contextual integrity for information-sharing and LLM delegation [46], [47].

The mechanism targets request-level purpose restriction for agent side effects. It turns the current user request into a short-lived, auditable purpose-like certificate that narrows an already-authorized agent tool surface; a statically authorized tool remains unavailable when the present request does not justify its effect. Unlike a declared-purpose database policy, this certificate is operational evidence for a bounded action decision, not proof that every downstream use of returned data satisfies a privacy purpose.

F. OpenPort Protocol

OpenPort Protocol provides the effect-control substrate for this paper [14]. It defines a governance-first gateway with authorization-dependent discovery, scoped permissions, data policy constraints, draft-first writes, preflight, state witness, idempotency, rate limits, and audit. The intent layer adds an upstream request envelope and explicitly measures the residual authority that effect controls leave in drafts.

XV. Conclusion

AI-agent access control must evaluate both credential permission and whether the proposed call is justified by the user's current expressed intent. This paper proposed a server-side intent-certificate authorization model with monotone session policy narrowing, intent-aware manifests, and consistency checks between user intent, tool choice, payload, and expected effect. OpenPort provides the existing effect-control substrate: static authorization, policy constraints, draft-first writes, preflight, state witnesses, idempotency, and audit. The intent-certificate layer performs request-level authorization before those effects are reached.

The evidence supports an executable request-envelope mechanism and static-policy non-expansion, while exposing a decisive limitation. In the real-model runtime study, the combined intent-layer and OpenPort path observes 0/198 completed unsafe effects but accepts 40/198 intent-inconsistent draft paths; strict benign completion is 0/108 and 36/108 benign trials produce governed artifacts. The 12-task transfer subset similarly observes 0/24 unsafe completed effects and 5/24 unsafe accepted drafts. Exact certificate bounds and adapter effect contracts remain the principal systems problem.

Appendix A
Reference Intent-Layer Reason Codes

- `agent.intent_not_found`
- `agent.intent_invalid`
- `agent.intent_binding_mismatch`
- `agent.intent_version_stale`
- `agent.intent_revoked`
- `agent.intent_low_confidence`
- `agent.intent_conflicting`
- `agent.intent_tool_mismatch`
- `agent.intent_payload_exceeds_bound`
- `agent.intent_expired`
- `agent.intent_review_required`

APPENDIX B
EXAMPLE TARGET-PROFILE CERTIFICATE

The following synthetic certificate illustrates the target reference profile used by the formal model. The target includes subject, message, canonicalization, policy, and registry-version bindings beyond the current in-memory prototype.

```
{
  "id": "intent_01",
  "profile": "intent-authorization-reference-v1",
  "requestHash": "sha256:...",
  "subjectBinding": {
    "actorId": "actor_demo",
    "appId": "app_demo",
    "keyId": "key_demo",
    "tenantId": "tenant_demo",
    "workspaceId": "workspace_demo",
    "sessionId": "session_demo",
    "messageId": "message_demo"
  },
  "intentClasses": ["summarize", "read"],
  "deniedIntentClasses": ["create", "update", "delete", "export", "admin"],
  "resourceBounds": {
    "resourceTypes": ["record"],
    "dateRange": { "start": "2026-06-10", "end": "2026-06-17" }
  },
  "effectBounds": {
    "maxRisk": "low",
    "allowExport": false,
    "allowMutation": false
  },
  "confidence": 0.86,
  "maxRoutingOutcome": "execute",
  "lifecycle": {
    "issuedAt": "2026-06-17T23:54:59Z",
    "expiresAt": "2026-06-17T23:59:59Z",
    "status": "active"
  },
  "versions": {
    "canonicalization": "canon-v1",
    "staticPolicy": "policy-demo-17",
    "toolRegistry": "registry-demo-4"
  },
  "classifierProvenance": {
    "source": "hybrid",
    "version": "hybrid-demo-v1"
  },
  "auditDigest": "sha256:..."
}
```

REFERENCES

[1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 471–10 506. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[3] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt Injection Attack to Tool Selection in LLM Agents," in *Network and Distributed System Security (NDSS) Symposium 2026*. San Diego, CA: Internet Society, Feb. 2026. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/prompt-injection-attack-to-tool-selection-in-llm-agents/

[4] J. Sneh, R. Yan, J. Yu, P. Torr, Y. Gal, S. Sengupta, E. Sommerlade, A. Paren, and A. Bibi, "ToolTweak: An Attack on Tool Selection in LLM-based Agents," 2025. [Online]. Available: https://arxiv.org/abs/2510.02554

[5] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, pp. 35 811–35 819, 2026. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/40895

[6] S. Jamshidi, A. Moradi Dakhel, K. W. Nafi, and F. Khomh, "Semantic Attacks on Tool-Augmented LLMs: Securing the Model Context Protocol Against Descriptor-Level Manipulation," 2025. [Online]. Available: https://arxiv.org/abs/2512.06556

[7] C. Huang, X. Huang, N. P. Tran, and A. M. Fard, "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning," 2026. [Online]. Available: https://arxiv.org/abs/2603.22489

[8] Y. T. Shen, K. Toyoda, and A. Leung, "MCP-38: A Comprehensive Threat Taxonomy for Model Context Protocol Systems (v1.0)," 2026. [Online]. Available: https://arxiv.org/abs/2603.18063

[9] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," https://genai.owasp.org/llm-top-10/, 2025, accessed 2026-06-17.

[10] ——, "LLM01:2025 Prompt Injection," https://genai.owasp.org/llmrisk/llm01-prompt-injection/, 2025, accessed 2026-06-17.
[11] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," National Institute of Standards and Technology, Gaithersburg, MD, NIST Trustworthy and Responsible AI 600-1, Jul. 2024. [Online]. Available: https://doi.org/10.6028/NIST.AI.600-1
[12] Model Context Protocol, "Authorization," https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization, Jun. 2025, specification version 2025-06-18. Accessed 2026-06-17.
[13] ——, "Tools," https://modelcontextprotocol.io/specification/2025-06-18/server/tools, Jun. 2025, specification version 2025-06-18. Accessed 2026-06-17.
[14] G. Zhu, C. Wang, Z. Wang, Z. Li, and Q. Li, "OpenPort Protocol: A Security Governance Specification for AI Agent Tool Access," 2026. [Online]. Available: https://arxiv.org/abs/2602.20196
[15] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975. [Online]. Available: https://doi.org/10.1109/PROC.1975.9939
[16] D. F. Ferraiolo and D. R. Kuhn, "Role-Based Access Controls," in *15th National Computer Security Conference*, Baltimore, MD, Oct. 1992, pp. 554–563. [Online]. Available: https://csrc.nist.gov/pubs/conference/1992/10/13/rolebased-access-controls/final
[17] V. C. Hu, D. Ferraiolo, D. R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-162, Jan. 2014, includes updates as of 2019-08-02. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162
[18] J. Park and R. Sandhu, "The UCON$_{ABC}$ Usage Control Model," *ACM Transactions on Information and System Security*, vol. 7, no. 1, pp. 128–174, Feb. 2004. [Online]. Available: https://doi.org/10.1145/984334.984339
[19] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Proceedings of the 2014 Network and Distributed System Security Symposium*. Internet Society, Feb. 2014. [Online]. Available: https://doi.org/10.14722/ndss.2014.23212
[20] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments?" 2026. [Online]. Available: https://arxiv.org/abs/2602.03117
[21] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities," in *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 1160–1183. [Online]. Available: https://aclanthology.org/2025.findings-naacl.65/
[22] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=roNSXZpUDN
[23] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *International Conference on Learning Representations*, 2024, spotlight. [Online]. Available: https://openreview.net/forum?id=GEcwtMk1uA
[24] W. Liu, Z. Liu, E. Dai, W. Yu, L. Yu, T. Yang, J. Han, and H. Gao, "MCPAgentBench: A Real-world Task Benchmark for Evaluating LLM Agent MCP Tool Use," 2025. [Online]. Available: https://arxiv.org/abs/2512.24565
[25] Y. Tang, T. Li, E. Li, C. J. Maddison, H. Dong, and Y. Ruan, "LM Agents May Fail to Act on Their Own Risk Knowledge," in *Conference on Language Modeling (COLM)*, 2025. [Online]. Available: https://openreview.net/forum?id=OeYdS51k8F
[26] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending Against Prompt Injection with Structured Queries," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 2383–2400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe
[27] T. Zhang, Y. Xu, J. Wang, K. Guo, X. Xu, B. Xiao, Q. Guan, J. Fan, J. Liu, Z. Liu, and H. Hu, "AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification," 2026. [Online]. Available: https://arxiv.org/abs/2602.22724
[28] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, "AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations," in *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, 2026, to appear. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/he-yu
[29] F. B. Schneider, "Enforceable Security Policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, Feb. 2000. [Online]. Available: https://doi.org/10.1145/353323.353382
[30] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2501.09674
[31] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A Security Architecture for Governing AI Agentic Systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.21034
[32] H. Gong, C. Li, R. Chang, and W. Shen, "Secure and Efficient Access Control for Computer-Use Agents via Context Space," 2025. [Online]. Available: https://arxiv.org/abs/2509.22256
[33] S. V. Kodathala, "aiAuthZ: Off-Host, Identity-Bound Authorization for AI Agents," 2026. [Online]. Available: https://arxiv.org/abs/2607.05518
[34] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 29 680–29 697. [Online]. Available: https://aclanthology.org/2025.acl-long.1435/
[35] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI Agents with Privilege Control," 2025. [Online]. Available: https://arxiv.org/abs/2504.11703
[36] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa, "MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents," 2025. [Online]. Available: https://arxiv.org/abs/2512.11147
[37] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," 2025. [Online]. Available: https://arxiv.org/abs/2503.18813
[38] J. Kim, W. Choi, and B. Lee, "Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents," 2025. [Online]. Available: https://arxiv.org/abs/2503.15547
[39] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. Rio de Janeiro, Brazil: ACM, Apr. 2026, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3744916.3764546

[40] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, ser. SOSP '97. New York, NY, USA: Association for Computing Machinery, Oct. 1997, pp. 129–142. [Online]. Available: https://doi.org/10.1145/268998.266669
[41] J.-W. Byun and N. Li, "Purpose Based Access Control for Privacy Protection in Relational Database Systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, Jul. 2008. [Online]. Available: https://doi.org/10.1007/s00778-006-0023-0
[42] K. Tran, S. Vasudevan, P. Desai, A. Gorelik, M. Ahuja, A. Yadatore Venkateshababu, M. Verma, D. Hu, W. E. Moustafa, V. Rajamani, A. Gupta, I. Buenrostro, K. Raina, and Y. Lin, "Data Guard: A Fine-grained Purpose-based Access Control System for Large Data Warehouses," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. Montreal, QC, Canada: IEEE, May 2026, official ICDE 2026 program entry; final IEEE DOI and page range were not available as of 2026-07-24. [Online]. Available: https://icde2026.github.io/program_details.html
[43] K. Wolf, F. Pallas, and S. Tai, "Messaging with Purpose Limitation – Privacy-Compliant Publish-Subscribe Systems," in *25th IEEE International Enterprise Distributed Object Computing Conference (EDOC)*, 2021, pp. 162–172.
[44] H. Nissenbaum, *Privacy in Context: Technology, Policy, and the Integrity of Social Life*. Stanford University Press, 2009.
[45] A. Barth, A. Datta, J. C. Mitchell, and H. Nissenbaum, "Privacy and Contextual Integrity: Framework and Applications," in *2006 IEEE Symposium on Security and Privacy*, 2006, pp. 184–198.
[46] S. Ghalebikesabi, E. Bagdasaryan, R. Yi, I. Yona, I. Shumailov, A. Pappu, C. Shi, L. Weidinger, R. Stanforth, L. Berrada, P. Kohli, P.-S. Huang, and B. Balle, "Operationalizing Contextual Integrity in Privacy-Conscious Assistants," 2024. [Online]. Available: https://arxiv.org/abs/2408.02373
[47] X. Huang, X. Cao, and W. Yang, "Need to Know: Contextual-Integrity-Grounded Query Rewriting for Privacy-Conscious LLM Delegation," 2026. [Online]. Available: https://arxiv.org/abs/2606.04067